Probability, Optimization, and Scientific Discovery

First-principles notes with differentiable stellar-physics examples from Stellax and the Jaxstro ecosystem

Anna Rosen
COMP 521 • San Diego State University

November 2026

These notes are for master's students in COMP 521 and for research students joining Jaxstro projects. They build one chain of reasoning: uncertain inputs become uncertain predictions; observations update what we know; derivatives tell us how to fit, how to sample, and what we can and cannot learn; and design tells us what to measure next. Every equation labelled TOY is a teaching construction, not a calibrated stellar-physics result. Every code listing was executed for these notes, and its printed output is shown beneath it.

Contents

How to use these notes **4**

1 Random variables and probability **7**

1.1 Deterministic physics, uncertain inputs 7
1.2 Discrete probabilities, densities, and the CDF 7
1.2.1 Worked example: a single power-law IMF 7
1.3 Expectation, variance, and standard error 8
1.4 Joint, conditional, and marginal distributions 8
1.4.1 Changing variables in a density 9
1.5 Three kinds of uncertainty, and one kind of error 9
1.6 Propagating uncertainty through a model 10
1.6.1 Worked example: uncertain mass, uncertain luminosity 10
1.6.2 Linear propagation in many dimensions 12
1.6.3 Other propagation methods 12
1.7 The law of total variance and shared physics 13

2 The language and mathematics of optimization **13**

2.1 Objective, loss, parameters, constraints 13

2.1.1 Two layers: solving the star and fitting the star . . . 14
2.2 From a noise model to a loss . . . 15
2.3 Gradient, Jacobian, Hessian . . . 15
2.3.1 The least-squares chain rule . . . 15
2.4 Gradient descent and the step size . . . 16
2.5 Newton, quasi-Newton, Gauss–Newton, Levenberg–Marquardt . . . 17
2.6 Minima, convexity, and conditioning . . . 20
2.7 Regularization, priors, and constraints . . . 21

3 From optimization to statistical inference 21
3.1 A likelihood is not a probability of the parameters . . . 21
3.2 Bayes' theorem . . . 21
3.2.1 The linear-Gaussian model, completely . . . 22
3.3 The Laplace approximation . . . 22
3.4 Prediction, nuisance parameters, and evidence . . . 23
3.5 Computing posteriors: Markov chain Monte Carlo . . . 24
3.6 Hierarchical models for stellar populations . . . 27
3.6.1 Selection effects . . . 28
3.7 Checking an inference code . . . 28

4 Identifiability and scientific discovery 30
4.1 Sensitivity is not identifiability . . . 30
4.2 Fisher information . . . 31
4.2.1 Fisher eigenvectors depend on your units . . . 32
4.3 Beyond local: profile likelihoods . . . 33
4.4 Model discrepancy and its own identifiability problem . . . 33
4.5 Optimal experimental design: what should we measure next? . . . 34
4.5.1 Three different optimization questions . . . 36
4.6 Surrogates, Gaussian processes, and multifidelity models . . . 36

5 Differentiable and trustworthy scientific computing 37
5.1 Automatic differentiation: JVPs and VJPs . . . 37
5.2 Differentiating through an implicit solve: the adjoint method . . . 37
5.3 Verifying a derivative . . . 39
5.4 When AD is confidently wrong . . . 41
5.5 Randomness and differentiability . . . 42

5.6 Parameters, hyperparameters, and training jargon 45

6 Two research workflows **46**

6.1 Workflow 1: how does uncertain opacity affect the TRGB? 46

6.2 Workflow 2: inferring an IMF slope from a cluster 47

6.3 A recommended progression for a research student 47

A Exercises **48**

B Worked solutions **50**

C Glossary **55**

How to use these notes

Audience and prerequisites. You should be comfortable with multivariable calculus (partial derivatives, the chain rule, Taylor expansion), linear algebra (matrix products, eigenvalues, solving linear systems), and Python with NumPy. You do not need prior coursework in statistics or optimization; Section 1 rebuilds the probability you need. Research students who already know that material can skim Section 1 and start at Section 1.6.

Glass box, then black box. Each algorithm appears first as a few lines of JAX that you could have written yourself: Levenberg–Marquardt, Hamiltonian Monte Carlo, simulation-based calibration, the adjoint method. Only after that do we point to the production library that does the same job more robustly. A library you can reproduce is a library you can debug.

Boxes. Four kinds of box recur.

Key idea

The conceptual point a section exists to make.

Common pitfall

A mistake that experienced researchers make, usually with confidence.

In Jaxstro

How the idea appears in the research codes: Stellax (differentiable stellar evolution), Gravax (differentiable N-body dynamics), Startrax (fast stellar tracks), Progenax (stellar-population initial conditions), and the observation and inference layers Fluxax and Informax. The code listings in these notes never call those packages: they run with plain JAX so that you can execute every one of them. The boxes explain what changes when the toy is replaced by the real solver.

Check your understanding

A short question to answer before reading on. The answer follows inside the box; cover it and commit to your own answer first. Longer practice problems are in Appendix A, with solutions in Appendix B.

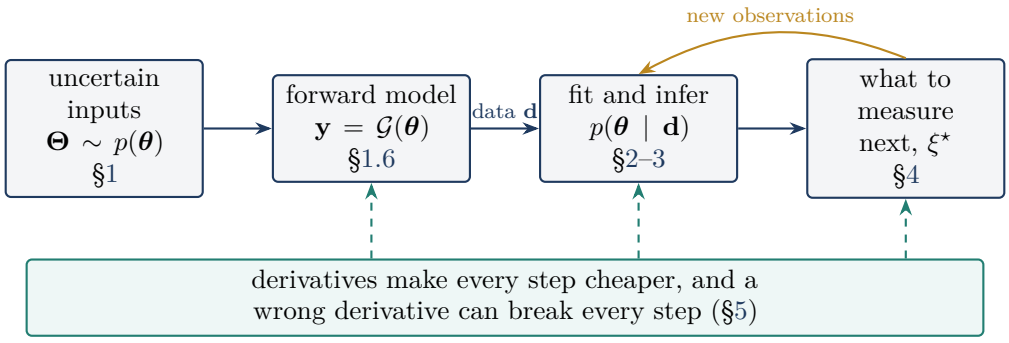
Running the code. Install JAX (`pip install "jax[cpu]"`); the listings were run with JAX 0.12 on a CPU. They enable 64-bit floats with `jax.config.update("jax_enable_x64", True)`. JAX defaults to 32-bit floats, which are fine for training neural networks but too coarse for derivative checks and ill-conditioned linear algebra (Section 5.3).

Learning goals

After working through these notes you should be able to:

1. Distinguish a deterministic model, a random variable, a probability distribution, and an uncertain physical parameter, and say which kind of uncertainty (aleatoric, epistemic, numerical) a given error bar represents.
2. Read notation such as $\arg\min$, $\nabla\mathcal{L}$, $\mathbf{J}$, $\mathbf{H}$, $p(\boldsymbol{\theta} \mid \mathbf{d})$ and $\mathbb{E}[X]$ aloud and explain it in words.
3. Propagate uncertainty through a nonlinear model by Monte Carlo and by first- and second-order linearization, and predict when each fails.
4. Turn a noise model into a loss, derive the gradient-descent, Newton, Gauss–Newton and Levenberg–Marquardt updates, and explain the role of conditioning.
5. Write Bayes' theorem for a stellar-physics problem, compute a posterior analytically in the linear-Gaussian case, by the Laplace approximation, and by Hamiltonian Monte Carlo.
6. Check an inference code with simulation-based calibration and diagnose sampler output with effective sample size.
7. Distinguish sensitivity from identifiability and structural from practical identifiability; use the Fisher matrix in a unit-independent way.
8. Quantify the value of a proposed observation with expected information gain.
9. Verify a derivative; compute gradients through implicit solves with the adjoint method; and recognize the two places automatic differentiation is confidently wrong: discontinuities in an expectation and event times in a fixed-step solver.
10. Design an emulator evaluation that does not leak information between training and test sets.

Road map



Optimization appears in every box: fitting parameters, training an emulator, tuning a sampler, choosing a design. The scientific question decides *what* is optimized; that choice is the part no library makes for you.

Notation

Each symbol below keeps this meaning throughout. A few letters also carry their standard astrophysical meaning in one place (for example κ for opacity in Section 6, ρ for mass density); those uses are flagged where they appear.

Symbol	Read aloud	Meaning
X, x	"capital X", "x"	a random variable and one of its realizations
$\boldsymbol{\theta}$, $\boldsymbol{\Theta}$	"theta"	parameters of a physical or statistical model; $\boldsymbol{\Theta}$ when treated as random
p	"p"	number of parameters, $\boldsymbol{\theta} \in \mathbb{R}^p$ (bold $\mathbf{p}$ in Section 3.5 is an HMC momentum)
$\mathcal{G}(\boldsymbol{\theta})$	"G of theta"	the forward model, e.g. a Stellax track mapped to observables
$\mathbf{y}$	"y"	a model prediction, $\mathbf{y} = \mathcal{G}(\boldsymbol{\theta})$
$\mathbf{d}$	"d" (bold)	observed data
ξ	"xi"	an experimental design we choose
$\mathcal{L}(\boldsymbol{\theta})$	"loss of theta"	a scalar objective to be *minimized*
$p(\mathbf{d} \mid \boldsymbol{\theta})$	"d given theta"	likelihood; $\ell(\boldsymbol{\theta}) = \log p(\mathbf{d} \mid \boldsymbol{\theta})$
$p(\boldsymbol{\theta} \mid \mathbf{d})$	"theta given d"	posterior
$U(\boldsymbol{\theta})$	"U of theta"	negative log posterior, $-\ell(\boldsymbol{\theta}) - \log p(\boldsymbol{\theta})$ up to a constant (the potential energy in HMC)
$\mathcal{U}(\xi)$	"utility of xi"	the score of a design ξ, e.g. its expected information gain
$\nabla\mathcal{L}$	"grad loss"	gradient vector of a scalar function
$\mathbf{J} = \partial\mathbf{y}/\partial\boldsymbol{\theta}$	"Jacobian"	matrix of first derivatives of a vector function
$\mathbf{H} = \nabla^2\mathcal{L}$	"Hessian"	matrix of second derivatives of a scalar function
$\mathcal{I}(\boldsymbol{\theta})$	"Fisher information"	expected curvature of $-\ell$
$F_X(x)$	"F of x"	cumulative distribution function of X
$\boldsymbol{\Sigma}$, $\boldsymbol{\Sigma}_0$	"Sigma", "Sigma nought"	noise covariance; prior covariance
$\mathbb{E}[X]$, $\mathrm{Var}(X)$	"expectation", "variance"	probability-weighted mean; mean squared deviation
$\mathbb{H}[X]$	"entropy of X"	entropy
η	"eta"	step size or learning rate
ν	"nu"	Levenberg–Marquardt damping
λ	"lambda"	regularization strength; λ_i with an index is an eigenvalue; bold $\boldsymbol{\lambda}$ is the adjoint vector of Section 5.2
$\boldsymbol{\Lambda}$	"capital lambda"	population-level hyperparameters (IMF slope, binary fraction, age spread)
$\boldsymbol{\phi}$	"phi"	nuisance parameters
$\boldsymbol{\psi}$	"psi"	stellar or dynamical physics shared by all stars in a population
$\mathbf{z}_i$	"z i"	star-specific latent variables (true mass, mass ratio, ...)
γ	"gamma"	slope of a power-law IMF, $\mathrm{d}N/\mathrm{d}m \propto m^{-\gamma}$
$\alpha(\boldsymbol{\Lambda}, \boldsymbol{\psi})$	"alpha of Lambda"	detection fraction: the probability that a star drawn from the population is detected (Section 3.6.1)
κ	"kappa"	condition number of a matrix (Section 2.4)
$\sim$, $\propto$	"distributed as", "proportional to"	declares a distribution; equality up to a constant factor

Vectors are bold, matrices are bold upright, and $^\mathsf{T}$ denotes a transpose. "log" is the natural logarithm, so information is measured in nats.

1 Random variables and probability

1.1 Deterministic physics, uncertain inputs

A Stellax run maps initial conditions and physics parameters to an evolutionary history,

$$\mathbf{y} = \mathcal{G}_{\text{Stellax}}(M, Z, t, \alpha_{\text{MLT}}, f_{\text{ov}}, \dots), \tag{1}$$

where M is the initial mass, Z the metallicity, t the age, α_{MLT} the mixing-length parameter, and f_{ov} an overshooting parameter. The output $\mathbf{y}$ might collect L, T_{eff}, radius, surface abundances, or the helium-core mass at the tip of the red-giant branch (TRGB). For fixed inputs and a fixed algorithm the output is fixed, up to numerical error.

Now pick a star at random from a cluster. You do not know its mass. Nothing about the physics has become random, yet the predicted luminosity is uncertain, because one of its inputs is. Probability is the bookkeeping for that situation.

Definition 1.1 (Random variable)**.** A *random variable* is a function $X : \Omega \to \mathbb{R}$ from a sample space Ω of possible outcomes to numbers.[1] A *realization* x is the value obtained in one outcome. A categorical outcome, such as remnant type, is a discrete random variable taking values in a finite set.

Key idea

Calling a quantity "random" is a statement about our information, not about the quantity. An uncertain mixing coefficient is modelled as a random variable because we do not know its value, not because it fluctuates from star to star.

1.2 Discrete probabilities, densities, and the CDF

A discrete random variable takes values in a countable set. A TOY remnant category might have $P(C = \text{WD}) = 0.70$, $P(C = \text{NS}) = 0.20$, $P(C = \text{BH}) = 0.10$: illustrative numbers that sum to one.

A continuous random variable such as mass has a *probability density function* (PDF) $p_M(m)$:

$$P(a \le M \le b) = \int_a^b p_M(m)\,\mathrm{d}m, \qquad \int_{-\infty}^{\infty} p_M(m)\,\mathrm{d}m = 1. \tag{2}$$

A density carries inverse units of its argument (here $M_\odot^{-1}$), so $p_M(1\,M_\odot)$ is *not* the probability that a star has exactly one solar mass; that probability is zero. Probability lives on intervals. The *cumulative distribution function* (CDF) is $F_M(m) = P(M \le m) = \int_{-\infty}^{m} p_M(u)\,\mathrm{d}u$; it rises monotonically from 0 to 1. A *quantile* inverts the CDF: the median $m_{1/2}$ satisfies $F_M(m_{1/2}) = 1/2$. The *mode* is where the density peaks. Mean, median and mode coincide for symmetric unimodal distributions (when the mean exists), and generally differ otherwise.

1.2.1 Worked example: a single power-law IMF

Progenax draws initial masses from an initial mass function (IMF). The simplest teaching model is

$$\frac{\mathrm{d}N}{\mathrm{d}m} \propto m^{-\gamma}, \qquad m \in [m_{\text{lo}}, m_{\text{hi}}]. \tag{3}$$

[1] Strictly, X must be measurable with respect to a probability measure on Ω; every function you will meet in these notes is.

$\mathrm{d}N/\mathrm{d}m$ counts stars per unit mass and is not normalized. (The astronomical literature often writes the IMF as $\xi(m)$; these notes reserve ξ for an experimental design.) The probability density is $p_M(m) = C\,m^{-\gamma}$ with

$$C = \frac{1-\gamma}{m_{\mathrm{hi}}^{1-\gamma} - m_{\mathrm{lo}}^{1-\gamma}} \quad (\gamma \neq 1), \qquad C = \frac{1}{\ln(m_{\mathrm{hi}}/m_{\mathrm{lo}})} \quad (\gamma = 1), \tag{4}$$

and CDF

$$F_M(m) = \frac{m^{1-\gamma} - m_{\mathrm{lo}}^{1-\gamma}}{m_{\mathrm{hi}}^{1-\gamma} - m_{\mathrm{lo}}^{1-\gamma}} \quad (\gamma \neq 1). \tag{5}$$

Salpeter's classic slope for massive stars is $\gamma \approx 2.35$ [1]. Realistic IMFs flatten below about $0.5\,M_\odot$ and are usually written as broken power laws or a lognormal joined to a power law [2, 3]; the single power law is here for its algebra.

Result 1.2 (Inverse-transform sampling)**.** If $U \sim \mathrm{Uniform}(0,1)$ and F is a continuous, strictly increasing CDF, then $M = F^{-1}(U)$ has CDF F. *Proof:* $P(M \le m) = P(F^{-1}(U) \le m) = P(U \le F(m)) = F(m)$, because F is increasing and U is uniform.

Solving Eq. (5) for m gives the sampler

$$M = F_M^{-1}(U;\gamma) = \left[m_{\mathrm{lo}}^{1-\gamma} + U\left(m_{\mathrm{hi}}^{1-\gamma} - m_{\mathrm{lo}}^{1-\gamma}\right)\right]^{1/(1-\gamma)}. \tag{6}$$

This produces a random population without making any stellar-evolution equation stochastic. It will also reappear in Section 5.5: because Eq. (6) is a smooth function of γ for fixed U, it lets us differentiate population statistics with respect to the IMF slope.

1.3 Expectation, variance, and standard error

The *expectation* of X is $\mathbb{E}[X] = \int x\,p_X(x)\,\mathrm{d}x$, and for any function g,

$$\mathbb{E}[g(X)] = \int g(x)\,p_X(x)\,\mathrm{d}x \tag{7}$$

(the "law of the unconscious statistician": you never need the density of $g(X)$ to average it). The *variance* is $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2] = \mathbb{E}[X^2] - (\mathbb{E}X)^2$, and the *standard deviation* is $\sigma_X = \sqrt{\mathrm{Var}\,X}$, in the units of X.

A *standard error* is a different object: the standard deviation of an *estimator*. For independent, identically distributed draws $x_1, \dots, x_N$ with variance σ^2, the sample mean $\bar{x} = N^{-1}\sum_i x_i$ has standard error $\sigma/\sqrt{N}$. This one formula governs every Monte Carlo estimate in these notes. Correlated draws, such as successive states of a Markov chain, break it; Section 3.5 shows how to repair it with an *effective* sample size.

1.4 Joint, conditional, and marginal distributions

Collect several uncertain inputs into a random vector, $\boldsymbol{\Theta} = (M, Z, \alpha_{\mathrm{MLT}}, f_{\mathrm{ov}}, f_{\mathrm{wind}})^{\mathsf{T}}$, where f_{wind} is a multiplier on a mass-loss prescription. A *joint* density $p(m, z)$ assigns density to combinations. The *conditional* density

$$p(m \mid z) = \frac{p(m,z)}{p(z)} \qquad (p(z) > 0) \tag{8}$$

describes mass once metallicity is known. *Marginalizing* integrates out what you do not keep: $p(m) = \int p(m, z)\,\mathrm{d}z$. Rearranging gives the product rule $p(m, z) = p(m \mid z)\,p(z)$, from which Bayes' theorem (Section 3) follows in one line.

Independence, $p(m, z) = p(m)p(z)$, is a strong claim. It does not follow from two parameters having different names. Uncertainties in opacity and in mixing efficiency are conceptually distinct, but if both were calibrated against the same solar model their *errors* are coupled.

The *covariance* is $\mathrm{Cov}(X_1, X_2) = \mathbb{E}[(X_1 - \mu_1)(X_2 - \mu_2)]$ and the correlation coefficient is $\rho_{12} = \mathrm{Cov}(X_1, X_2)/(\sigma_1\sigma_2) \in [-1, 1]$. Zero correlation does not imply independence: if $X \sim \mathcal{N}(0, 1)$ then X and X^2 are uncorrelated but completely dependent. For a linear combination $Y = \mathbf{u}^\mathsf{T}\boldsymbol{X}$ with covariance matrix $\boldsymbol{\Sigma}_X$,

$$\mathrm{Var}(Y) = \mathbf{u}^\mathsf{T}\boldsymbol{\Sigma}_X\mathbf{u}, \qquad \text{e.g. } \mathrm{Var}(aX_1 + bX_2) = a^2\sigma_1^2 + b^2\sigma_2^2 + 2ab\ \mathrm{Cov}(X_1, X_2). \tag{9}$$

The cross term is where compensating stellar prescriptions hide: a high opacity and a high mixing efficiency can push T_{eff} in opposite directions.

1.4.1 Changing variables in a density

If $Y = g(X)$ with g smooth and invertible, probability is conserved interval by interval, $p_Y(y)\,|\mathrm{d}y| = p_X(x)\,|\mathrm{d}x|$, so

$$p_Y(y) = p_X\big(g^{-1}(y)\big)\left|\frac{\mathrm{d}g^{-1}}{\mathrm{d}y}\right|, \qquad \text{and in } n \text{ dimensions} \qquad p_{\boldsymbol{Y}}(\boldsymbol{y}) = p_{\boldsymbol{X}}(\boldsymbol{x})\,\big|\det \partial\boldsymbol{x}/\partial\boldsymbol{y}\big|. \tag{10}$$

The Jacobian factor is the reason a density, unlike a probability, changes when you change units or coordinates. It explains why MAP estimates are not invariant under reparameterization (Section 3.2) and why log-parameters need a correction term when sampling (Section 3.5).

Worked example 1.3 (From an IMF to a luminosity function)**.** Take the IMF of Eq. (4) and the TOY relation $L = M^4$ (solar units). Then $m = L^{1/4}$ and $\mathrm{d}m/\mathrm{d}L = \frac{1}{4}L^{-3/4}$, so

$$p_L(L) = C\,(L^{1/4})^{-\gamma} \cdot \tfrac{1}{4}L^{-3/4} = \tfrac{C}{4}\,L^{-(\gamma+3)/4}.$$

A Salpeter IMF ($\gamma = 2.35$) becomes a luminosity function with slope -1.34. Forgetting the Jacobian factor would give slope -0.59: an error in the shape of the distribution, not just its normalization.

1.5 Three kinds of uncertainty, and one kind of error

Aleatoric uncertainty is variability of outcomes under a fixed conditioning set: draw a different star, get a different mass; repeat an exposure, get a different photon count. *Epistemic* uncertainty is missing knowledge: the true value of a reaction rate, a mixing coefficient, or the form of a missing physical process. Both are represented with probability, but they behave differently: more stars shrink the uncertainty in an average over stars; more stars do not, by themselves, teach you a shared physics parameter (Section 1.7). Which label applies depends on what is conditioned on.

Numerical error is different in kind. Unconverged integration, a poorly placed mesh, a bug, or a wrong derivative is not a random variable with a known distribution. It is removed by convergence studies, conserved quantities, reference solutions, and derivative checks, not averaged away.

Source	Stellar example	Scientific response
Population variability	the mass of the next Progenax draw	sample an explicit population model
Measurement noise	photometric scatter	a defensible likelihood with its covariance
Unknown shared physics	the opacity or mixing law in Stellax	parameters with priors; calibration; tests against independent data
Model inadequacy	missing transport or mass-loss physics	improve the model, or add an explicit discrepancy term (Section 4.4)
Numerical error	unconverged solve; wrong event sensitivity	verify the computation before any inference

Common pitfall

A narrow posterior is not a correct posterior. If the noise model is wrong or a physical process is missing, more data make the error bars smaller around the wrong answer.

1.6 Propagating uncertainty through a model

If $\boldsymbol{\Theta} \sim p(\boldsymbol{\theta})$ and $\boldsymbol{Y} = \mathcal{G}(\boldsymbol{\Theta})$, the distribution of $\boldsymbol{Y}$ is the *pushforward* of $p(\boldsymbol{\theta})$ through $\mathcal{G}$. You rarely need its density; you need to sample it or compute its moments. Three strategies follow.

1.6.1 Worked example: uncertain mass, uncertain luminosity

Take the TOY relation $L/L_\odot = (M/M_\odot)^4$ and $X = M/M_\odot \sim \mathcal{N}(1, 0.05^2)$. (A Gaussian allows negative masses; here that is a 20σ event and harmless, but a general model would truncate.)

First-order (delta method). Linearize: $L \approx f(\mu) + f'(\mu)(X - \mu)$. Then $\mathbb{E}[L] \approx f(\mu) = 1$ and $\sigma_L \approx |f'(\mu)|\sigma = 4 \times 0.05 = 0.20$; equivalently $\sigma_L/L \approx |\mathrm{d}\log L/\mathrm{d}\log M|\,\sigma_M/M = 20\%$.

Exact moments. For a Gaussian, $\mathbb{E}[X^4] = \mu^4 + 6\mu^2\sigma^2 + 3\sigma^4 = 1.01501875$, and the exact standard deviation (from $\mathbb{E}[X^8]$) is 0.2026. So $\mathbb{E}[f(X)] \neq f(\mathbb{E}[X])$: averaging the prediction is not predicting at the average. The shift is set by curvature.

Second-order. Expanding one term further,

$$\mathbb{E}[f(X)] \approx f(\mu) + \tfrac{1}{2} f''(\mu)\,\sigma^2, \tag{11}$$

gives $1 + \frac{1}{2} \cdot 12 \cdot 0.0025 = 1.015$, recovering all but 1.9×10^{-5} of the exact mean (the remaining $3\sigma^4$ is fourth order). In several dimensions the correction is $\frac{1}{2}\,\mathrm{tr}(\mathbf{H}_{\mathcal{G}}\boldsymbol{\Sigma}_\Theta)$ per output.

Monte Carlo. Draw $\boldsymbol{\theta}^{(i)} \sim p(\boldsymbol{\theta})$, evaluate $\mathbf{y}^{(i)} = \mathcal{G}(\boldsymbol{\theta}^{(i)})$, and average: $\widehat{\mathbb{E}[\boldsymbol{Y}]} = N^{-1}\sum_i \mathbf{y}^{(i)}$, with standard error $\sigma_Y/\sqrt{N}$. Monte Carlo needs no derivatives and handles skewed, multimodal and discontinuous outputs, but its error falls only as $N^{-1/2}$, and rare regimes (for example the few stars that reach core collapse) need importance sampling or stratification to be resolved at all.

ex01_propagation.py -- the three strategies side by side, plus a pathwise gradient

```python
"""Uncertainty propagation through the toy relation L/Lsun = (M/Msun)^4.

Compares (i) Monte Carlo, (ii) first-order delta method, (iii) second-order
delta method, and the pathwise gradient of E[L] with respect to the mean mass.
"""
import jax
import jax.numpy as jnp

jax.config.update("jax_enable_x64", True)  # derivative checks need float64


def lum(m):  # toy forward model, valid only in a limited mass range
    return m**4


mu, sigma = 1.0, 0.05

# (i) Monte Carlo with a FIXED set of standard-normal draws (common random numbers)
eps = jax.random.normal(jax.random.key(0), (1_000_000,))


def mc_mean_lum(mu, sigma):
    return jnp.mean(lum(mu + sigma * eps))  # reparameterization: M = mu + sigma*eps


L_mc = lum(mu + sigma * eps)
print(f"MC     : E[L] = {L_mc.mean():.5f}  sd[L] = {L_mc.std():.5f}"
      f"  (MC std. error of mean = {L_mc.std() / jnp.sqrt(eps.size):.1e})")

# (ii) first-order delta method: mean = f(mu), sd = |f'(mu)| sigma
d1 = jax.grad(lum)(mu)
d2 = jax.grad(jax.grad(lum))(mu)
print(f"Delta-1: E[L] = {lum(mu):.5f}  sd[L] = {abs(d1) * sigma:.5f}")

# (iii) second-order correction to the mean: + (1/2) f''(mu) sigma^2
print(f"Delta-2: E[L] = {lum(mu) + 0.5 * d2 * sigma**2:.5f}")

# Exact Gaussian moments for comparison
exact_mean = mu**4 + 6 * mu**2 * sigma**2 + 3 * sigma**4
print(f"Exact  : E[L] = {exact_mean:.8f}")
z = (L_mc.mean() - exact_mean) / (L_mc.std() / jnp.sqrt(eps.size))
print(f"MC error in units of its standard error: z = {z:+.2f}")

# Pathwise gradient of the sample-average objective vs the exact derivative
g_path = jax.grad(mc_mean_lum, argnums=0)(mu, sigma)
g_exact = 4 * mu**3 + 12 * mu * sigma**2
print(f"dE[L]/dmu: pathwise = {g_path:.5f}   exact = {g_exact:.5f}")
```

Output (executed for these notes):

```
MC     : E[L] = 1.01500  sd[L] = 0.20258  (MC std. error of mean = 2.0e-04)
Delta-1: E[L] = 1.00000  sd[L] = 0.20000
Delta-2: E[L] = 1.01500
Exact  : E[L] = 1.01501875
MC error in units of its standard error: z = -0.11
dE[L]/dmu: pathwise = 4.02994   exact = 4.03000
```

Two habits are worth copying from this listing. First, the Monte Carlo result is reported with its

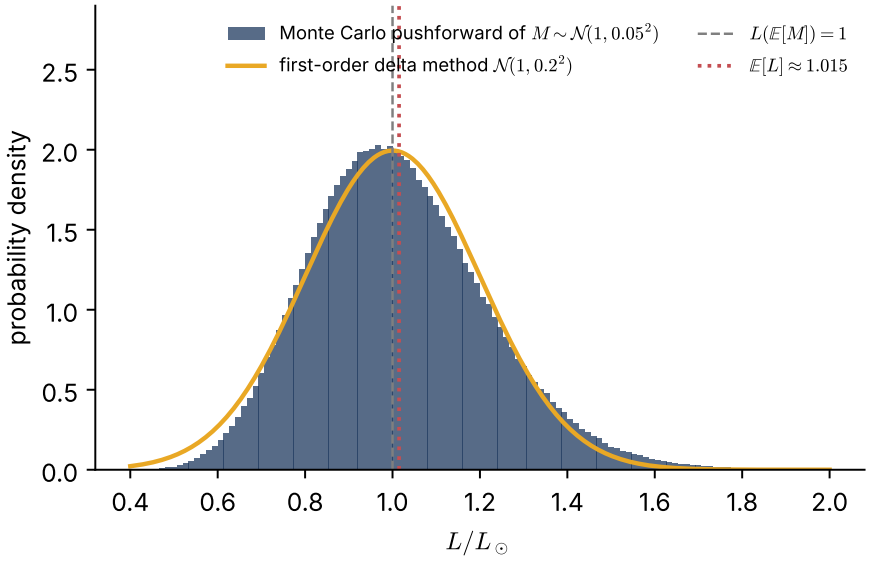


Figure 1: A deterministic nonlinear map turns a symmetric input distribution into a skewed output. The first-order delta method (gold) gets the width nearly right but misses the skew and the shift of the mean. Data are synthetic.

standard error and compared in units of it ($z = -0.11$); a Monte Carlo number without an error bar is not a result. Second, the gradient of the Monte Carlo average with respect to μ is taken with the standard-normal draws held fixed; Section 5.5 explains why that is legitimate here and when it is not.

1.6.2 Linear propagation in many dimensions

Linearizing about the mean $\boldsymbol{\mu}$, $\mathcal{G}(\boldsymbol{\Theta}) \approx \mathcal{G}(\boldsymbol{\mu}) + \mathbf{J}(\boldsymbol{\Theta} - \boldsymbol{\mu})$ with $\mathbf{J} = \partial \mathbf{y}/\partial \boldsymbol{\theta}|_{\boldsymbol{\mu}}$, gives

$$\boldsymbol{\Sigma}_Y \approx \mathbf{J}\, \boldsymbol{\Sigma}_\Theta\, \mathbf{J}^{\mathsf{T}}. \tag{12}$$

With Stellax and JAX, $\mathbf{J}$ costs a handful of forward- or reverse-mode passes (Section 5.1) instead of thousands of tracks, which is one of the main practical reasons to build a differentiable stellar code. The price is that Eq. (12) assumes $\mathcal{G}$ is close to linear over the width of $p(\boldsymbol{\theta})$.

Where linearization breaks

The delta method fails when the input distribution straddles a qualitative change: helium-flash versus quiescent helium ignition, the mass that separates white-dwarf from neutron-star progenitors, a change of convective-core boundary. A Jacobian evaluated on one side knows nothing about the other. Always validate Eq. (12) against a targeted ensemble near such boundaries.

1.6.3 Other propagation methods

Quasi-Monte Carlo replaces random draws with low-discrepancy point sets (for example Sobol sequences); for smooth integrands of modest effective dimension its error falls closer to N^{-1}.

Polynomial chaos expansion fits the model with polynomials orthogonal under $p(\boldsymbol{\theta})$ and reads moments off the coefficients. Neither is efficient across sharp discontinuities or in very high effective dimension.

Check your understanding

Without computing anything: for $L = M^4$, is $\mathbb{E}[L]$ above or below $L(\mathbb{E}[M])$, and why? Would the answer change for $L \propto M^{0.5}$?
Answer. Above: M^4 is convex, so by Jensen's inequality $\mathbb{E}[f(X)] \geq f(\mathbb{E}X)$. For the concave $M^{0.5}$ the inequality reverses.

1.7 The law of total variance and shared physics

For any random Y and $\boldsymbol{\Theta}$,

$$\mathrm{Var}(Y) = \underbrace{\mathbb{E}_{\Theta}\left[\mathrm{Var}(Y \mid \boldsymbol{\Theta})\right]}_{\text{scatter at fixed physics}} + \underbrace{\mathrm{Var}_{\Theta}\left(\mathbb{E}[Y \mid \boldsymbol{\Theta}]\right)}_{\text{spread caused by uncertain physics}} . \tag{13}$$

This splits uncertainty into the part that remains even if the physics were known and the part caused by not knowing it.

Worked example 1.4 (A shared systematic sets a floor)**.** Let $Y_i = \theta + \epsilon_i$ for $i = 1, \ldots, N$, with an uncertain offset shared by all stars, $\mathrm{Var}(\theta) = \sigma_\theta^2$, and independent scatter $\mathrm{Var}(\epsilon_i) = \sigma_\epsilon^2$. For the sample mean $\bar{Y}$, conditioning on θ gives $\mathrm{Var}(\bar{Y} \mid \theta) = \sigma_\epsilon^2/N$ and $\mathbb{E}[\bar{Y} \mid \theta] = \theta$, so Eq. (13) yields

$$\mathrm{Var}(\bar{Y}) = \frac{\sigma_\epsilon^2}{N} + \sigma_\theta^2.$$

With $\sigma_\theta = 0.1$, $\sigma_\epsilon = 0.2$ and $N = 100$, $\mathrm{sd}(\bar{Y}) = \sqrt{0.0004 + 0.01} \approx 0.102$, not the $\sqrt{0.05/100} \approx 0.022$ you would get by treating all noise as independent. Be careful which variance you mean: this is the variance of the *sample mean* $\bar{Y}$. A single star's Y_i has variance $\sigma_\theta^2 + \sigma_\epsilon^2$, which does not depend on N at all.

More stars cannot average away a shared parameter, but more *kinds* of data can constrain it, if the model and data make it identifiable. That is the subject of Section 4.

2 The language and mathematics of optimization

2.1 Objective, loss, parameters, constraints

An optimization problem chooses parameters to minimize a scalar objective over a feasible set $\mathcal{F}$:

$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta} \in \mathcal{F}} \mathcal{L}(\boldsymbol{\theta}). \tag{14}$$

$\min_{\boldsymbol{\theta}} \mathcal{L}$ is the smallest *value*; $\arg\min_{\boldsymbol{\theta}} \mathcal{L}$ is the *location* achieving it. For $\mathcal{L}(\theta) = (\theta - 3)^2 + 2$, the minimum is 2 and the argmin is 3. With ties, the argmin is a set; libraries return one element of it, which is where degeneracies hide.

A *decision variable* is something the optimizer adjusts; a *state variable* describes the system and is computed from the decision variables. In a Stellax calibration, an opacity correction a is a

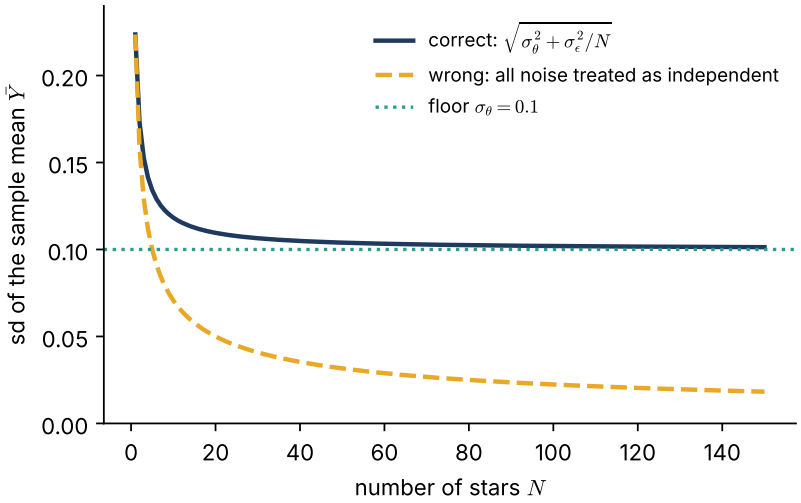


Figure 2: Independent scatter averages down as $N^{-1/2}$; an unconstrained shared offset does not.

decision variable; the profiles $T(r)$, $\rho(r)$ and abundances $X_i(r)$ are state variables computed by the stellar-structure solver. A *loss* is an objective penalizing misfit; a *regularized objective* adds penalties encoding preferences. The objective encodes what we told the optimizer to value, not what nature values.

2.1.1 Two layers: solving the star and fitting the star

Stellar structure defines the state implicitly through a residual equation

$$\mathbf{R}(\mathbf{s}, \boldsymbol{\theta}) = \mathbf{0}, \tag{15}$$

solved by Newton iterations inside Stellax. This is *root-finding*. Fitting observations is an *outer* optimization over $\boldsymbol{\theta}$, each evaluation of which runs the inner solve. Keep the layers separate. Writing $\|\mathbf{R}\|^2$ as one more term in the outer loss turns exact conservation laws into soft, tunable penalties, and a small residual does not guarantee you are on the physical solution branch.

Worked example 2.1 (A constrained calibration, stated in the right layer)**.** Calibrate $\boldsymbol{\theta} = (\alpha_{\mathrm{MLT}}, f_{\mathrm{ov}}, X_0, Y_0, Z_0)$, where the last three are the *initial* hydrogen, helium and metal mass fractions:

$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}} \mathcal{L}_{\mathrm{data}}(\boldsymbol{\theta}) \quad \text{subject to} \quad f_{\mathrm{ov}} \geq 0, \quad X_0 + Y_0 + Z_0 = 1, \quad X_0, Y_0, Z_0 \geq 0.$$

An *active* constraint holds with equality at the optimum (for example $f_{\mathrm{ov}}^\star = 0$).

Constraints belong in the right layer

Only the *initial* composition is a calibration parameter. The abundances $X_i(r)$ inside the model star are state variables: the identities $\sum_i X_i(r) = 1$ and $X_i(r) \geq 0$ must hold at every mesh point and every timestep, so they are the solver's job, enforced by construction (for example by evolving all species with a conservative reaction network). Imposing them as constraints on the outer fit would mix the two layers.

2.2 From a noise model to a loss

Let the residuals be $\mathbf{r}(\boldsymbol{\theta}) = \mathcal{G}(\boldsymbol{\theta}) - \mathbf{d}$ (some authors use the opposite sign). The loss should come from a model of how the data were generated, not from habit.

Gaussian noise. If $\mathbf{d} \sim \mathcal{N}(\mathcal{G}(\boldsymbol{\theta}), \boldsymbol{\Sigma})$ with $\boldsymbol{\Sigma}$ independent of $\boldsymbol{\theta}$, the negative log-likelihood is, up to a constant,

$$\mathcal{L}(\boldsymbol{\theta}) = -\ell(\boldsymbol{\theta}) = \tfrac{1}{2}\mathbf{r}^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\mathbf{r}. \tag{16}$$

The inverse covariance down-weights noisy and redundant data. For independent errors $\boldsymbol{\Sigma}$ is diagonal and $\mathcal{L} = \frac{1}{2}\sum_i (r_i/\sigma_i)^2 = \frac{1}{2}\chi^2$. In code it is cleanest to *whiten*: with a Cholesky factor $\boldsymbol{\Sigma} = \mathbf{C}\mathbf{C}^{\mathsf{T}}$, define $\tilde{\mathbf{r}} = \mathbf{C}^{-1}\mathbf{r}$, so $\mathcal{L} = \frac{1}{2}\|\tilde{\mathbf{r}}\|^2$ and all later formulas hold with $\boldsymbol{\Sigma} = \mathbf{I}$.

Parameter-dependent noise

If the covariance depends on parameters (a fitted "jitter" term, or model-dependent theoretical errors), the negative log-likelihood is $\frac{1}{2}\mathbf{r}^{\mathsf{T}}\boldsymbol{\Sigma}(\boldsymbol{\theta})^{-1}\mathbf{r} + \frac{1}{2}\log\det\boldsymbol{\Sigma}(\boldsymbol{\theta}) + \text{const}$. Dropping the log-determinant lets the optimizer reduce χ^2 by inflating the errors without limit.

Counts. For photon or star counts n_i with model expectations $\mu_i(\boldsymbol{\theta})$, the Poisson likelihood gives $\mathcal{L} = \sum_i [\mu_i - n_i \log \mu_i]$ (plus a constant). Replacing it with χ^2 biases fits wherever counts are small. Censored data, upper limits, and contaminated samples each need their own likelihood.

Worked example 2.2 (A complete TOY calibration)**.** Let a normalized observable obey the TOY local relation $y(\theta) = 1 + 2\theta$, and observe $d = 3$ with $\sigma = 0.5$. Then $\mathcal{L}(\theta) = \frac{1}{2}\big((1+2\theta-3)/0.5\big)^2 = 8(\theta-1)^2$, with minimizer $\theta^\star = 1$ and minimum 0. We use this loss to study algorithms before substituting an expensive Stellax calculation.

2.3 Gradient, Jacobian, Hessian

For a scalar $\mathcal{L}(\boldsymbol{\theta})$ with p parameters, the *gradient* $\nabla\mathcal{L} \in \mathbb{R}^p$ collects $\partial\mathcal{L}/\partial\theta_j$. It points in the direction of steepest increase *for the Euclidean metric in the coordinates you chose*; rescale a parameter and the steepest direction changes. For a vector function $\mathcal{G} : \mathbb{R}^p \to \mathbb{R}^m$, the *Jacobian* is the $m \times p$ matrix $J_{ij} = \partial\mathcal{G}_i/\partial\theta_j$. For a scalar function, the *Hessian* $H_{ij} = \partial^2\mathcal{L}/\partial\theta_i\partial\theta_j$ describes curvature. A zero gradient with positive-definite Hessian guarantees a strict local minimum; a zero gradient alone could be a minimum, maximum, or saddle.

Dimensions are a free check. If Stellax maps 5 parameters to 20 observables, $\mathbf{J}$ is 20×5, but the gradient of a scalar loss built from those observables has length 5.

2.3.1 The least-squares chain rule

For $\mathcal{L} = \frac{1}{2}\mathbf{r}^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\mathbf{r}$ with fixed $\boldsymbol{\Sigma}$ and $\mathbf{J} = \partial\mathbf{r}/\partial\boldsymbol{\theta}$,

$$\nabla\mathcal{L} = \mathbf{J}^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\mathbf{r}, \qquad \mathbf{H} = \underbrace{\mathbf{J}^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\mathbf{J}}_{\mathbf{H}_{\mathrm{GN}}} + \sum_i (\boldsymbol{\Sigma}^{-1}\mathbf{r})_i \, \nabla^2 r_i. \tag{17}$$

The *Gauss–Newton* matrix $\mathbf{H}_{\mathrm{GN}}$ drops the second term, which is small when residuals are small or the model is nearly linear. It is always positive semi-definite, needs only first derivatives, and, as

Section 4.2 shows, is exactly the Fisher information. Remember the matrix $\mathbf{J}^\top\boldsymbol{\Sigma}^{-1}\mathbf{J}$: it appears three more times.

2.4 Gradient descent and the step size

Gradient descent steps against the gradient:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k \nabla\mathcal{L}(\boldsymbol{\theta}_k), \tag{18}$$

with step size (learning rate) η_k. For $\mathcal{L} = 8(\theta-1)^2$, $\mathcal{L}' = 16(\theta-1)$, so $\theta_{k+1} - 1 = (1-16\eta)(\theta_k - 1)$. The error shrinks iff $|1-16\eta| < 1$, i.e. $0 < \eta < 1/8$; $\eta = 1/16$ lands on the minimum in one step; $\eta = 1/8$ oscillates forever between two points.

The general lesson: conditioning sets the speed. Near a minimum every smooth loss looks like a quadratic $\frac{1}{2}\boldsymbol{e}^\top\mathbf{H}\boldsymbol{e}$ in the error $\boldsymbol{e} = \boldsymbol{\theta} - \boldsymbol{\theta}^\star$. Then $\boldsymbol{e}_{k+1} = (\mathbf{I} - \eta\mathbf{H})\boldsymbol{e}_k$, and along each eigenvector of $\mathbf{H}$ (eigenvalue λ_i) the error is multiplied by $1 - \eta\lambda_i$ per step. Hence:

- stability requires $\eta < 2/\lambda_{\max}$: the stiffest direction caps the step;
- the best fixed step, $\eta = 2/(\lambda_{\min} + \lambda_{\max})$, contracts the error by $(\kappa-1)/(\kappa+1)$ per iteration, where $\kappa = \lambda_{\max}/\lambda_{\min}$ is the condition number;
- reaching a tolerance therefore takes roughly $\frac{\kappa}{2}\ln(1/\text{tol})$ iterations.

ex03_conditioning.py -- iterations needed grow linearly with the condition number

```python
"""Gradient descent on an ill-conditioned quadratic, and what rescaling does.

L(theta) = 1/2 theta^T H theta with H = diag(1, kappa).  For fixed step eta,
GD converges iff eta < 2/lambda_max; the best fixed step gives the error
contraction factor (kappa - 1)/(kappa + 1) per iteration.
"""
import jax
import jax.numpy as jnp

jax.config.update("jax_enable_x64", True)


def gd_iterations(kappa, tol=1e-6, max_iter=100_000):
    H = jnp.diag(jnp.array([1.0, kappa]))
    loss = lambda th: 0.5 * th @ H @ th
    grad = jax.jit(jax.grad(loss))
    eta = 2.0 / (1.0 + kappa)  # optimal fixed step for this quadratic
    th = jnp.array([1.0, 1.0])
    for k in range(max_iter):
        if jnp.linalg.norm(th) < tol:
            return k
        th = th - eta * grad(th)
    return max_iter


for kappa in [1.0, 10.0, 100.0, 1000.0]:
    print(f"kappa = {kappa:7.0f}: GD needs {gd_iterations(kappa):6d} iterations;"
          f" predicted contraction per step = {(kappa - 1) / (kappa + 1):.4f}")

```

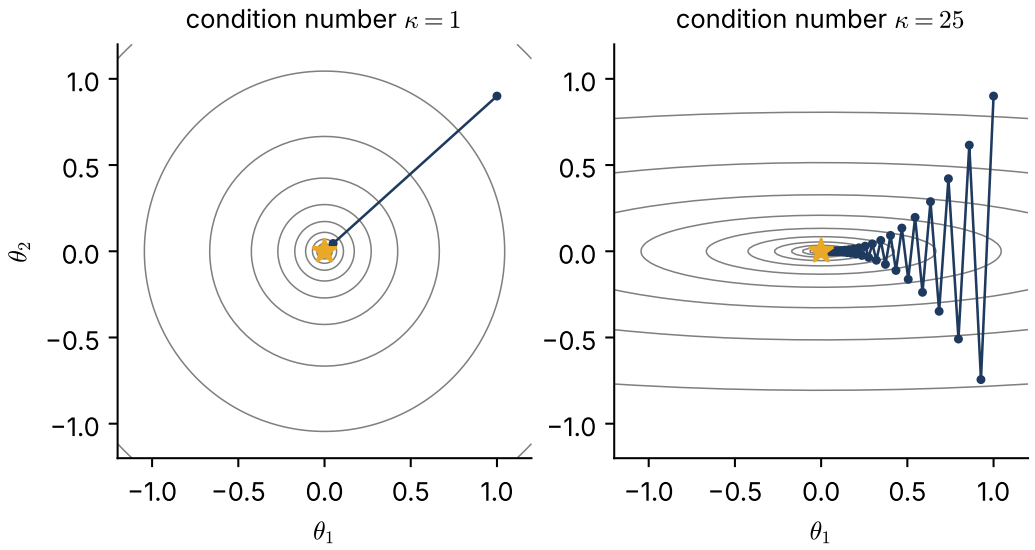


Figure 3: Gradient descent with a near-optimal fixed step. Left: a round bowl ($\kappa = 1$). Right: an elongated valley ($\kappa = 25$); the step is limited by the steep direction, so progress along the shallow direction is slow and the iterates zig-zag.

```python
# Newton's method solves H d = -g and lands on the minimum of a quadratic in one step
H = jnp.diag(jnp.array([1.0, 1000.0]))
th0 = jnp.array([1.0, 1.0])
th1 = th0 + jnp.linalg.solve(H, -(H @ th0))
print("Newton step from (1,1):", th1)

# Rescaling z = sqrt(H) theta turns the kappa=1000 problem into kappa=1:
# in z-coordinates the Hessian is D^-1 H D^-1 with D = sqrt(H)
D_inv = jnp.diag(1.0 / jnp.sqrt(jnp.diag(H)))
print(f"cond(H) = {jnp.linalg.cond(H):.1f} -> after diagonal rescaling:"
      f" {jnp.linalg.cond(D_inv @ H @ D_inv):.1f}")
```

Output (executed for these notes):

```
kappa =       1: GD needs      1 iterations; predicted contraction per step = 0.0000
kappa =      10: GD needs     71 iterations; predicted contraction per step = 0.8182
kappa =     100: GD needs    709 iterations; predicted contraction per step = 0.9802
kappa =    1000: GD needs   7082 iterations; predicted contraction per step = 0.9980
Newton step from (1,1): [0. 0.]
cond(H) = 1000.0 -> after diagonal rescaling: 1.0
```

Momentum methods (heavy ball, Nesterov acceleration) improve the dependence to roughly $\sqrt{\kappa}$. An *iteration* is one update. An *epoch*, in machine learning, is one pass through a finite training set, which may contain many iterations when data are processed in mini-batches.

2.5 Newton, quasi-Newton, Gauss–Newton, Levenberg–Marquardt

Newton's method minimizes the local quadratic model $\mathcal{L}(\boldsymbol{\theta} + \boldsymbol{\Delta}) \approx \mathcal{L} + \mathbf{g}^\mathsf{T}\boldsymbol{\Delta} + \frac{1}{2}\boldsymbol{\Delta}^\mathsf{T}\mathbf{H}\boldsymbol{\Delta}$ by solving

$$\mathbf{H}\boldsymbol{\Delta} = -\mathbf{g}, \qquad \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \boldsymbol{\Delta}. \tag{19}$$

Solve the linear system; never form $\mathbf{H}^{-1}$. On a quadratic, Newton converges in one step regardless of κ (last lines of Listing 2). Far from a minimum, $\mathbf{H}$ may be indefinite and the Newton step can

point uphill, so practical methods add safeguards:

- **Line search** picks a step length along a direction, accepting it only if the loss falls enough (the Armijo condition $\mathcal{L}(\boldsymbol{\theta} + t\boldsymbol{\Delta}) \leq \mathcal{L}(\boldsymbol{\theta}) + c\, t\, \mathbf{g}^\mathsf{T}\boldsymbol{\Delta}$, $c \approx 10^{-4}$).
- **Trust regions** trust the quadratic model only within a radius, which grows when predicted and actual decreases agree and shrinks when they do not.
- **Quasi-Newton** methods (BFGS, L-BFGS) build curvature estimates from successive gradients. They are strong defaults for smooth, deterministic objectives of moderate dimension.
- **Gauss–Newton** replaces $\mathbf{H}$ by $\mathbf{H}_{\text{GN}} = \mathbf{J}^\mathsf{T}\boldsymbol{\Sigma}^{-1}\mathbf{J}$ for least squares.
- **Levenberg–Marquardt** (LM) damps Gauss–Newton,

$$(\mathbf{J}^\mathsf{T}\boldsymbol{\Sigma}^{-1}\mathbf{J} + \nu\,\mathbf{D})\boldsymbol{\Delta} = -\mathbf{J}^\mathsf{T}\boldsymbol{\Sigma}^{-1}\mathbf{r}, \tag{20}$$

  with $\mathbf{D}$ the identity or $\operatorname{diag}(\mathbf{H}_{\text{GN}})$. Large ν gives a short, scaled gradient step; small ν gives Gauss–Newton. Adjusting ν after each accepted or rejected step is a simple trust-region strategy.

ex04_levenberg_marquardt.py -- LM from scratch on a TOY cooling curve

```python
"""Levenberg-Marquardt from first principles on a toy cooling curve.

Model (toy): y(t; theta) = exp(a) * exp(-t / exp(b)),  theta = (a, b) = (log L0, log tau).
Log parameters keep L0 and tau positive and make their scales comparable.
Shows that J^T Sigma^-1 J is simultaneously the Gauss-Newton Hessian, the Fisher
matrix, and (plus the prior precision) the Laplace-approximation posterior precision.
"""
import jax
import jax.numpy as jnp

jax.config.update("jax_enable_x64", True)

t = jnp.linspace(0.0, 4.0, 12)
theta_true = jnp.array([jnp.log(2.0), jnp.log(1.5)])
sigma = 0.05 * jnp.ones_like(t)


def model(theta):
    a, b = theta
    return jnp.exp(a) * jnp.exp(-t / jnp.exp(b))


d = model(theta_true) + sigma * jax.random.normal(jax.random.key(3), t.shape)


def residual(theta):  # whitened residuals r / sigma, so Sigma^-1 is the identity below
    return (model(theta) - d) / sigma


def loss(theta):
    r = residual(theta)
    return 0.5 * r @ r


jac = jax.jacfwd(residual)  # 12 x 2 Jacobian of whitened residuals
```

```
36
37 theta, nu = jnp.array([0.0, 0.0]), 1e-2  # start far off; nu = damping
38 for it in range(50):
39     r, J = residual(theta), jac(theta)
40     g, H_gn = J.T @ r, J.T @ J
41     step = jnp.linalg.solve(H_gn + nu * jnp.diag(jnp.diag(H_gn)), -g)
42     if loss(theta + step) < loss(theta):
43         theta, nu = theta + step, nu / 3.0  # accept: trust the quadratic model more
44     else:
45         nu = nu * 5.0  # reject: damp toward (scaled) gradient descent
46     if jnp.linalg.norm(g) < 1e-6:  # small gradient: stop
47         break
48
49 J = jac(theta)
50 F = J.T @ J  # Fisher information for Gaussian noise with fixed covariance
51 H_full = jax.hessian(loss)(theta)
52 print(f"stopped after {it + 1} iterations: theta = {theta}, truth = {theta_true}")
53 print(f"chi^2 at optimum = {2 * loss(theta):.2f} for {t.size - 2} degrees of freedom")
54 print("Gauss-Newton Hessian / Fisher:\n", F)
55 print("Full Hessian (adds sum_i r_i * Hess(r_i)):\n", H_full)
56
57 prior_cov = jnp.diag(jnp.array([1.0, 1.0]))  # N(0, 1) priors on a and b
58 post_cov = jnp.linalg.inv(F + jnp.linalg.inv(prior_cov))  # Laplace approximation
59 print("Laplace posterior sd:", jnp.sqrt(jnp.diag(post_cov)))
60 print("posterior correlation of (a, b):",
61       post_cov[0, 1] / jnp.sqrt(post_cov[0, 0] * post_cov[1, 1]))
```

Output (executed for these notes):

```
stopped after 11 iterations: theta = [0.69581024 0.4158928 ], truth = [0.69314718 0.40546511]
chi^2 at optimum = 7.23 for 10 degrees of freedom
Gauss-Newton Hessian / Fisher:
 [[4207.41458128 1600.84176474]
 [1600.84176474 1530.17117584]]
Full Hessian (adds sum_i r_i * Hess(r_i)):
 [[4207.41458128 1600.84176483]
 [1600.84176483 1490.16330579]]
Laplace posterior sd: [0.0198625  0.03292921]
posterior correlation of (a, b): -0.6306338457586195
```

Read the output closely. The fit recovers the truth to within its uncertainty. $\chi^2 = 7.2$ for 10 degrees of freedom is a reasonable fit. The Gauss–Newton matrix agrees with the full Hessian in the (a, a) and (a, b) entries (to the precision of the stopping rule) and differs in the (b, b) entry by 3%. The exact agreement is a consequence of how the amplitude enters the model, not of luck; Exercise 8 asks you to prove it. The 3% difference is small because the residuals at the optimum are small, which is when Gauss–Newton is a good approximation. The Laplace posterior (previewed here, derived in Section 3.3) shows the two parameters are anticorrelated: a brighter, faster-cooling curve and a fainter, slower one fit nearly equally well.

Other optimizers. *Adam* rescales each coordinate by running estimates of gradient moments; it is designed for noisy mini-batch gradients in neural-network training and is usually *worse* than L-BFGS or LM for smooth stellar-parameter fits. *CMA-ES* adapts a population distribution and needs no gradients, which makes it a robust fallback when gradients are unavailable or untrustworthy, at a cost of many more model evaluations. In the JAX ecosystem, Optimistix provides Newton root-finding, quasi-Newton (BFGS), Gauss–Newton and LM solvers and Optax provides first-order optimizers such as Adam.

Situation	Reasonable first choice	Why
smooth least squares, $p \lesssim 10^3$	LM or Gauss–Newton with trust region	uses residual structure
smooth general loss, moderate p	L-BFGS with line search	curvature from gradients only
huge p, noisy mini-batch gradients	Adam or SGD with momentum	cheap, tolerant of noise
gradients unavailable or wrong	CMA-ES, Nelder–Mead	no derivatives needed
several basins suspected	any of the above from many starts	local methods find local minima

Choose by smoothness, dimension, derivative validity and constraints, then measure convergence. Do not choose by fashion.

2.6 Minima, convexity, and conditioning

A *local minimum* is no worse than nearby feasible points; a *global minimum* is no worse than any feasible point. A function is *convex* if the chord between any two points of its graph lies *on or above* the graph. For differentiable f on a convex domain this is equivalent to

$$f(\boldsymbol{\theta}') \geq f(\boldsymbol{\theta}) + \nabla f(\boldsymbol{\theta})^{\mathsf{T}}(\boldsymbol{\theta}' - \boldsymbol{\theta}) \quad \text{for all } \boldsymbol{\theta}, \boldsymbol{\theta}', \tag{21}$$

so every tangent plane is a *global* under-estimator. The word "global" is the point: near a strict local minimum, every smooth function satisfies such a bound locally, and that guarantees nothing about other basins. Consequently a convex function on a convex set has no local minima that are not global (the minimizer need not be unique).

Astrophysical losses are usually non-convex, with several basins and long, flat valleys. A tiny gradient may mean a minimum, a saddle, or a direction the data do not constrain at all.

Conditioning measures how much a solution moves when its inputs are perturbed. If two columns of $\mathbf{J}$ are nearly parallel, different parameter combinations produce nearly the same observables, and $\mathbf{J}^{\mathsf{T}}\mathbf{J}$ is ill-conditioned: its condition number is the *square* of that of $\mathbf{J}$, which is one reason good least-squares codes factor $\mathbf{J}$ (QR or SVD) instead of forming $\mathbf{J}^{\mathsf{T}}\mathbf{J}$.

Choose coordinates deliberately. Parameters with very different units or scales should be nondimensionalized, $z_i = (\theta_i - \theta_{0,i})/s_i$, or logged, $z = \log\theta$, for positive quantities (as in Listing 3). This changes the numerical geometry, and when the ill-conditioning comes purely from mismatched scales (as in Listing 2) it can turn a $\kappa = 1000$ problem into $\kappa = 1$. It cannot create information the data lack. When the objective is a log-posterior, which is a density over $\boldsymbol{\theta}$, changing coordinates also adds the log-Jacobian of Eq. (10). A log-likelihood is a density in $\mathbf{d}$, not in $\boldsymbol{\theta}$, so it gets no Jacobian term (Section 3).

Convergence is not validation

Stopping rules (small gradient, small step, small relative decrease) are statements about the algorithm. They certify neither global optimality, nor physical correctness, nor identifiability. Validation means checking numerical accuracy, derivatives, residual structure, physical assumptions, and predictions for data not used in the fit.

2.7 Regularization, priors, and constraints

Regularization adds preferences to a fit, for example $\mathcal{J}(\boldsymbol{\theta}) = \mathcal{L}_{\text{data}}(\boldsymbol{\theta}) + \lambda\|\boldsymbol{\theta}\|_2^2$. The penalty depends on parameter scales and is not a physical law.

There is an exact Bayesian reading. A Gaussian prior $p(\boldsymbol{\theta}) \propto \exp(-\frac{1}{2}\boldsymbol{\theta}^\mathsf{T}\boldsymbol{\Sigma}_0^{-1}\boldsymbol{\theta})$ adds $\frac{1}{2}\boldsymbol{\theta}^\mathsf{T}\boldsymbol{\Sigma}_0^{-1}\boldsymbol{\theta}$ to the negative log-likelihood, so ridge regression with strength λ is MAP estimation with $\boldsymbol{\Sigma}_0 = (2\lambda)^{-1}\mathbf{I}$. An ℓ_1 penalty $\lambda\sum_i|\theta_i|$ is MAP with a Laplace prior. Note the asymmetry: the ℓ_1 *MAP estimate* is often exactly sparse, but the posterior *mean* under the same prior is not. "Regularization" and "prior" are the same mathematics; writing the prior makes the assumption visible and lets you propagate it.

Hard constraints have their own toolbox. *Projection* maps each iterate back into the feasible set (easy for bounds). *Reparameterization* removes the constraint ($f_{\text{ov}} = e^z$; a softmax for mass fractions). It has a cost: e^z approaches the bound but never reaches it, so if the constraint is active at the optimum ($f_{\text{ov}}^\star = 0$), the optimizer drifts toward $z \to -\infty$ instead of stopping. *Barrier* methods add terms that blow up at the boundary. *Lagrange multipliers* turn constraints into stationarity conditions (the KKT conditions for inequalities). *ADMM* splits certain structured problems into easier subproblems; it carries convergence guarantees for convex problems, not for nonconvex coupled physics.

3 From optimization to statistical inference

3.1 A likelihood is not a probability of the parameters

Given data $\mathbf{d}$, the *likelihood* is the sampling model for the data, read as a function of the parameters:

$$\boldsymbol{\theta} \mapsto p(\mathbf{d} \mid \boldsymbol{\theta}), \qquad \ell(\boldsymbol{\theta}) = \log p(\mathbf{d} \mid \boldsymbol{\theta}). \tag{22}$$

It is a density in $\mathbf{d}$, not in $\boldsymbol{\theta}$: integrating it over $\boldsymbol{\theta}$ does not generally give one, and it has no meaning as "the probability that $\boldsymbol{\theta}$ is true". Its negative logarithm is a loss, and minimizing it gives the *maximum-likelihood estimate* $\hat{\boldsymbol{\theta}}_{\text{ML}} = \arg\max_{\boldsymbol{\theta}} p(\mathbf{d} \mid \boldsymbol{\theta}) = \arg\min_{\boldsymbol{\theta}}[-\ell(\boldsymbol{\theta})]$.

The MLE has a useful invariance: if $\boldsymbol{\varphi} = g(\boldsymbol{\theta})$ is a reparameterization, then $\hat{\boldsymbol{\varphi}}_{\text{ML}} = g(\hat{\boldsymbol{\theta}}_{\text{ML}})$, because a likelihood value is attached to a parameter *point* and carries no Jacobian. The MLE is a point; it says nothing about uncertainty on its own.

3.2 Bayes' theorem

The product rule written both ways, $p(\boldsymbol{\theta}, \mathbf{d}) = p(\mathbf{d} \mid \boldsymbol{\theta})p(\boldsymbol{\theta}) = p(\boldsymbol{\theta} \mid \mathbf{d})p(\mathbf{d})$, gives

$$\underbrace{p(\boldsymbol{\theta} \mid \mathbf{d})}_{\text{posterior}} = \frac{\overbrace{p(\mathbf{d} \mid \boldsymbol{\theta})}^{\text{likelihood}}\ \overbrace{p(\boldsymbol{\theta})}^{\text{prior}}}{\underbrace{p(\mathbf{d})}_{\text{evidence}}}, \qquad p(\mathbf{d}) = \int p(\mathbf{d} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta})\, \mathrm{d}\boldsymbol{\theta}. \tag{23}$$

The posterior is a full distribution over parameters. Its mode is the *maximum a posteriori* (MAP) estimate, $\boldsymbol{\theta}_{\text{MAP}} = \arg\min_{\boldsymbol{\theta}}[-\ell(\boldsymbol{\theta}) - \log p(\boldsymbol{\theta})]$, which is exactly the regularized fit of Section 2.7.

The MAP depends on your coordinates

A posterior *density* transforms with the Jacobian of Eq. (10), so its peak moves when you change variables. Example: if $p(\theta \mid \mathbf{d}) \propto \theta e^{-\theta}$ for $\theta > 0$, the MAP is $\theta = 1$. In $z = \log\theta$ the density is $p(z \mid \mathbf{d}) \propto \theta^2 e^{-\theta}$, whose peak is at $\theta = 2$. The posterior *probability* of any interval is the same in both coordinates; only the summary changed. Report posterior medians and quantile-based (equal-tailed) intervals, which do transform consistently, when the parameterization is arbitrary.

3.2.1 The linear-Gaussian model, completely

Let $\theta \sim \mathcal{N}(\mu_0, \tau_0^2)$ and $y = a\theta + \epsilon$ with $\epsilon \sim \mathcal{N}(0, \sigma^2)$. The log posterior is a quadratic in θ,

$$\log p(\theta \mid y) = -\frac{(\theta - \mu_0)^2}{2\tau_0^2} - \frac{(y - a\theta)^2}{2\sigma^2} + \text{const},$$

and completing the square gives a Gaussian posterior whose *precisions add*:

$$\frac{1}{\tau_{\text{post}}^2} = \frac{1}{\tau_0^2} + \frac{a^2}{\sigma^2}, \qquad \mu_{\text{post}} = \tau_{\text{post}}^2 \left(\frac{\mu_0}{\tau_0^2} + \frac{a\,y}{\sigma^2} \right). \tag{24}$$

The posterior mean is a precision-weighted average of the prior mean and the value y/a implied by the data. For a vector $\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0)$ and $\mathbf{y} = \mathbf{J}\boldsymbol{\theta} + \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$:

$$\boldsymbol{\Sigma}_{\text{post}} = (\boldsymbol{\Sigma}_0^{-1} + \mathbf{J}^\top \boldsymbol{\Sigma}^{-1} \mathbf{J})^{-1}, \qquad \boldsymbol{\mu}_{\text{post}} = \boldsymbol{\Sigma}_{\text{post}} (\boldsymbol{\Sigma}_0^{-1} \boldsymbol{\mu}_0 + \mathbf{J}^\top \boldsymbol{\Sigma}^{-1} \mathbf{y}). \tag{25}$$

There is $\mathbf{J}^\top \boldsymbol{\Sigma}^{-1} \mathbf{J}$ again, this time as the precision the data contribute.

Worked example 3.1 (Updating a standardized opacity correction)**.** Let $\mu_0 = 0$, $\tau_0 = 1$, $a = 2$, $\sigma = 1$, and observe $y = 1$. Then $1/\tau_{\text{post}}^2 = 1 + 4 = 5$, so $\tau_{\text{post}}^2 = 0.2$, and $\mu_{\text{post}} = 0.2 \times (0 + 2 \times 1/1) = 0.4$:

$$\theta \mid y = 1 \sim \mathcal{N}(0.4,\, 0.2).$$

The standard deviation fell from 1 to $\sqrt{0.2} \approx 0.447$. The central 95% credible interval is $0.4 \pm 1.96\sqrt{0.2} \approx [-0.48, 1.28]$.

A *credible interval* contains a stated posterior probability, conditional on the model and prior. A frequentist *confidence interval* is produced by a procedure that covers the true value in a stated fraction of repeated experiments. They coincide in the linear-Gaussian model with a flat prior and differ in general; Section 3.7 shows how to test whether a Bayesian procedure has frequentist coverage.

3.3 The Laplace approximation

Most posteriors are not Gaussian, but many are nearly Gaussian near their peak. Expand the negative log posterior $U(\boldsymbol{\theta}) = -\ell(\boldsymbol{\theta}) - \log p(\boldsymbol{\theta})$ to second order about the MAP:

$$p(\boldsymbol{\theta} \mid \mathbf{d}) \approx \mathcal{N}(\boldsymbol{\theta}_{\text{MAP}},\, \mathbf{H}_U^{-1}), \qquad \mathbf{H}_U = \nabla^2 U(\boldsymbol{\theta}_{\text{MAP}}) \approx \mathbf{J}^\top \boldsymbol{\Sigma}^{-1} \mathbf{J} + \boldsymbol{\Sigma}_0^{-1} \tag{26}$$

for Gaussian noise and a Gaussian prior, using the Gauss–Newton approximation of Eq. (17). This is what Listing 3 computed, with one shortcut: it centred the Gaussian on the maximum-likelihood fit rather than on the MAP. With $\mathcal{N}(0, 1)$ priors and a data precision in the thousands, the two differ by far less than the posterior width. It costs one optimization plus one Hessian, and it fails for skewed, bounded, or multimodal posteriors, in which case you sample (Section 3.5).

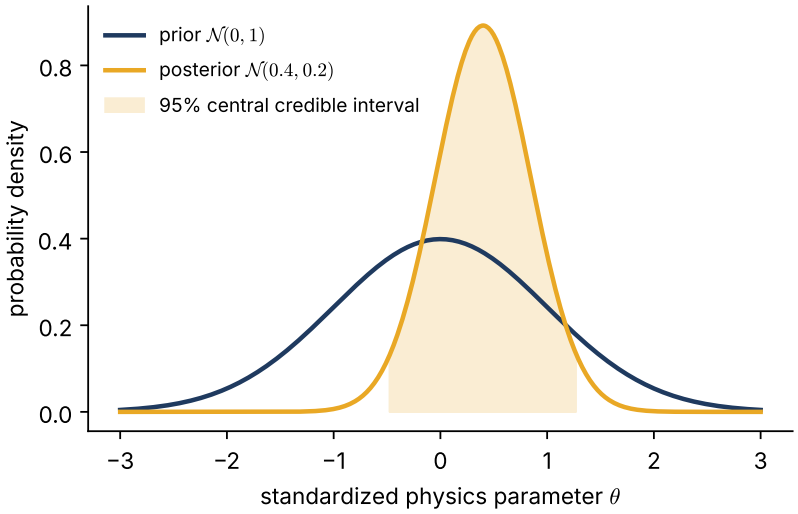


Figure 4: Bayesian updating in the linear-Gaussian TOY model: the posterior is narrower than the prior and shifted toward the value the data favour.

One matrix, four jobs

For Gaussian noise with fixed covariance, the single matrix $\mathbf{J}^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\mathbf{J}$ is

1. the **Gauss–Newton Hessian** that sets optimizer steps (Eq. (17));
2. the **precision added by the data** in the linear-Gaussian posterior (Eq. (25)) and the Laplace approximation (Eq. (26));
3. the **Fisher information**, whose inverse bounds estimator variance (Section 4.2);
4. the matrix that sets the **expected information gain**, through $\det(\mathbf{I}+\boldsymbol{\Sigma}_0\mathcal{I})$, of a design (Section 4.5).

If you understand this matrix for your problem, you understand how hard it is to fit, how well it is constrained, and what would constrain it better.

3.4 Prediction, nuisance parameters, and evidence

To predict a new observable $\tilde{\mathbf{d}}$, average over what you do not know:

$$p(\tilde{\mathbf{d}} \mid \mathbf{d}) = \int p(\tilde{\mathbf{d}} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta} \mid \mathbf{d})\, \mathrm{d}\boldsymbol{\theta}. \tag{27}$$

This *posterior predictive distribution* combines observational scatter with parameter uncertainty; plugging in $\boldsymbol{\theta}_{\mathrm{MAP}}$ keeps only the first and understates the spread.

A *nuisance parameter* is needed by the data model but is not the scientific target. To constrain an exotic energy-loss coupling g, the conventional opacity, mixing, age, composition and calibration parameters $\boldsymbol{\phi}$ are nuisances. Bayesian analysis *marginalizes* them, $p(g \mid \mathbf{d}) = \int p(g, \boldsymbol{\phi} \mid \mathbf{d})\, \mathrm{d}\boldsymbol{\phi}$. Frequentist analysis often *profiles*:

$$\ell_{\mathrm{prof}}(g) = \max_{\boldsymbol{\phi}} \ell(g, \boldsymbol{\phi}). \tag{28}$$

The two answer different questions and give different intervals when the posterior in $\boldsymbol{\phi}$ is skewed or its width depends on g.

In Jaxstro

A constraint on axion-like cooling from TRGB luminosities means something only if the conventional stellar uncertainties, observational error, selection, and competing explanations are in the model as nuisance parameters. A best-fitting nonzero coupling, on its own, is not evidence for a new particle.

The evidence and its prior sensitivity. The evidence $p(\mathbf{d})$ normalizes the posterior and compares models through Bayes factors $p(\mathbf{d} \mid \mathcal{M}_1)/p(\mathbf{d} \mid \mathcal{M}_2)$. It is much more sensitive to the prior than the posterior is. For a parameter with a uniform prior of width W and a likelihood of width $\sigma_L \ll W$, $p(\mathbf{d}) \approx p(\mathbf{d} \mid \hat{\theta})\sqrt{2\pi}\,\sigma_L/W$. So if $\mathcal{M}_2$ adds this parameter to $\mathcal{M}_1$, doubling its arbitrary prior width halves $p(\mathbf{d} \mid \mathcal{M}_2)$ and doubles the Bayes factor in favour of the simpler $\mathcal{M}_1$, although the fit is unchanged. Bayes factors computed with vague priors therefore need prior-sensitivity checks.

3.5 Computing posteriors: Markov chain Monte Carlo

Beyond conjugate and Laplace cases, we represent the posterior by *samples*: with draws $\boldsymbol{\theta}^{(1)}, \ldots, \boldsymbol{\theta}^{(N)}$ any posterior expectation is a Monte Carlo average. Markov chain Monte Carlo (MCMC) builds a chain whose long-run distribution is the posterior, and needs the posterior only up to the unknown constant $p(\mathbf{d})$.

Metropolis. From the current $\boldsymbol{\theta}$, propose $\boldsymbol{\theta}' = \boldsymbol{\theta} + \boldsymbol{\epsilon}$ from a symmetric distribution and accept with probability $\min\{1,\ p(\boldsymbol{\theta}' \mid \mathbf{d})/p(\boldsymbol{\theta} \mid \mathbf{d})\}$; otherwise stay. This satisfies *detailed balance*, $p(\boldsymbol{\theta} \mid \mathbf{d})\,T(\boldsymbol{\theta} \to \boldsymbol{\theta}') = p(\boldsymbol{\theta}' \mid \mathbf{d})\,T(\boldsymbol{\theta}' \to \boldsymbol{\theta})$, where T is the probability of moving between the two states in one step, so the posterior is stationary. The evidence cancels in the ratio. In a long narrow valley, a random-walk proposal must be as small as the valley is narrow, and then it takes many steps to diffuse along the valley.

Hamiltonian Monte Carlo. HMC [4, 5] adds an auxiliary momentum $\mathbf{p} \sim \mathcal{N}(\mathbf{0}, \mathbf{M})$ and simulates the dynamics of a particle with potential energy $U(\boldsymbol{\theta}) = -\log p(\boldsymbol{\theta} \mid \mathbf{d})$,

$$\mathcal{H}(\boldsymbol{\theta}, \mathbf{p}) = U(\boldsymbol{\theta}) + \tfrac{1}{2}\mathbf{p}^{\mathsf{T}}\mathbf{M}^{-1}\mathbf{p}, \qquad \dot{\boldsymbol{\theta}} = \mathbf{M}^{-1}\mathbf{p}, \quad \dot{\mathbf{p}} = -\nabla U(\boldsymbol{\theta}). \tag{29}$$

The *leapfrog* integrator (half kick, drift, half kick) is time-reversible and preserves phase-space volume, so the Metropolis acceptance probability reduces to $\min\{1, e^{-\Delta\mathcal{H}}\}$, with $\Delta\mathcal{H}$ the energy error of the numerical trajectory. A trajectory of tens of steps can cross the whole posterior along the valley while still being accepted with high probability. This is a central reason to make stellar models differentiable: ∇U requires $\nabla_{\boldsymbol{\theta}}\mathcal{G}$. The No-U-Turn Sampler (NUTS) chooses the trajectory length automatically [6].

ex06_hmc.py -- random-walk Metropolis and HMC, from scratch

```python
"""Random-walk Metropolis vs Hamiltonian Monte Carlo, written from scratch in JAX.

Target: a strongly correlated 2-D Gaussian (correlation 0.98), a stand-in for a
posterior with a near-degeneracy between two stellar-physics parameters.
"""
import jax
import jax.numpy as jnp

jax.config.update("jax_enable_x64", True)

rho = 0.98
Cov = jnp.array([[1.0, rho], [rho, 1.0]])
Prec = jnp.linalg.inv(Cov)


def log_post(x):  # unnormalized log density
    return -0.5 * x @ Prec @ x


grad_log_post = jax.grad(log_post)


def rwm_step(x, key, step=0.25):
    k1, k2 = jax.random.split(key)
    prop = x + step * jax.random.normal(k1, x.shape)
    accept = jnp.log(jax.random.uniform(k2)) < log_post(prop) - log_post(x)
    return jnp.where(accept, prop, x), accept


def leapfrog(x, p, eps, n_steps):
    p = p + 0.5 * eps * grad_log_post(x)
    def body(_, xp):
        x, p = xp
        x = x + eps * p
        return x, p + eps * grad_log_post(x)
    x, p = jax.lax.fori_loop(0, n_steps - 1, body, (x, p))
    x = x + eps * p
    return x, p + 0.5 * eps * grad_log_post(x)


def hmc_step(x, key, eps=0.15, n_steps=20):
    k1, k2 = jax.random.split(key)
    p0 = jax.random.normal(k1, x.shape)  # identity mass matrix
    x1, p1 = leapfrog(x, p0, eps, n_steps)
    log_r = (log_post(x1) - 0.5 * p1 @ p1) - (log_post(x) - 0.5 * p0 @ p0)  # -dH
    accept = jnp.log(jax.random.uniform(k2)) < log_r
    return jnp.where(accept, x1, x), accept


def run(step_fn, key, n=4000, x0=jnp.array([2.0, -2.0])):
    def body(x, k):
        x, a = step_fn(x, k)
        return x, (x, a)
    _, (xs, acc) = jax.lax.scan(body, x0, jax.random.split(key, n))
    return xs, acc.mean()


```

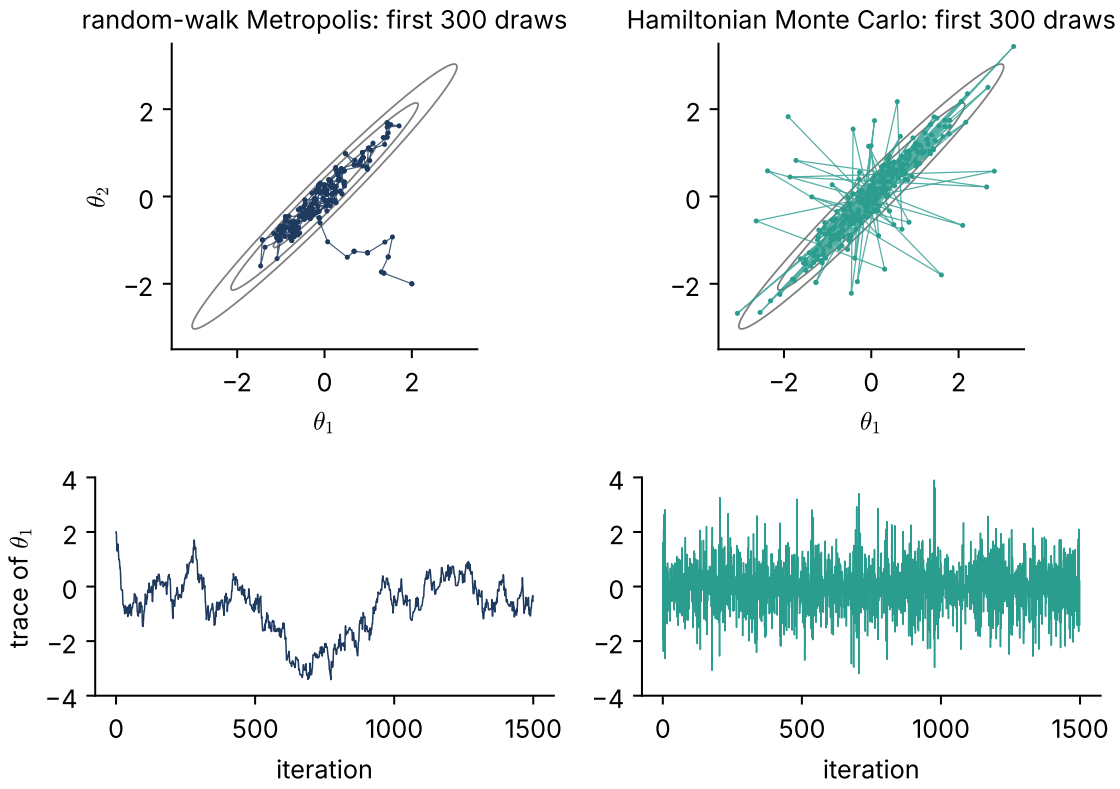


Figure 5: Sampling a correlation-0.98 Gaussian, a stand-in for a near-degenerate pair of physics parameters. Top: the first 300 draws. Bottom: traces. Random-walk Metropolis (left) diffuses slowly along the valley; HMC (right) crosses it in one trajectory.

```python
def ess(chain):
    """Effective sample size with Geyer's initial-positive-sequence truncation."""
    c = chain - chain.mean()
    acf = jnp.correlate(c, c, mode="full")[c.size - 1:] / (c @ c)
    pairs = acf[0:-1:2] + acf[1::2]  # Gamma_k = rho_{2k} + rho_{2k+1}
    keep = jnp.cumprod(pairs > 0)  # stop at the first non-positive pair
    tau = -1.0 + 2.0 * jnp.sum(pairs * keep)  # integrated autocorrelation time
    return chain.size / tau


for name, fn in [("RWM", rwm_step), ("HMC", hmc_step)]:
    xs, acc = run(jax.jit(fn), jax.random.key(7))
    xs = xs[500:]  # discard warm-up
    print(f"{name}: acceptance {acc:.2f}, mean {xs.mean(0)}, sd {xs.std(0)},"
          f" ESS(x1) ~ {ess(xs[:, 0]):.0f} of {xs.shape[0]}")
```

Output (executed for these notes):

```
RWM: acceptance 0.55, mean [-0.30896375 -0.31172623], sd [1.04482784 1.04035183], ESS(x1) ~ 14 of 3500
HMC: acceptance 0.96, mean [-0.0109746  -0.01198857], sd [0.98852799 0.9932576 ], ESS(x1) ~ 10112 of 3500
```

Reading sampler output. Successive MCMC draws are correlated, so the standard error of a posterior mean is $\sigma/\sqrt{N_{\text{eff}}}$ with *effective sample size* $N_{\text{eff}} = N/\tau_{\text{int}}$ and $\tau_{\text{int}} = 1 + 2\sum_{k\geq 1}\rho_k$ the integrated autocorrelation time. The listing estimates τ_{int} by summing autocorrelations in pairs until

a pair turns negative (Geyer's initial positive sequence). Random-walk Metropolis delivers about 14 effective draws from 3500, which is why its mean is off by 0.3: the target has unit standard deviation, so the standard error is $1/\sqrt{14} \approx 0.27$ and the error is about 1σ. HMC reports $N_{\text{eff}} \approx 10^4 > N$. That is not a bug: its successive draws are *anti*-correlated (lag-1 autocorrelation ≈ -0.5), and anticorrelated draws estimate a mean better than independent ones.

Always check, in this order:

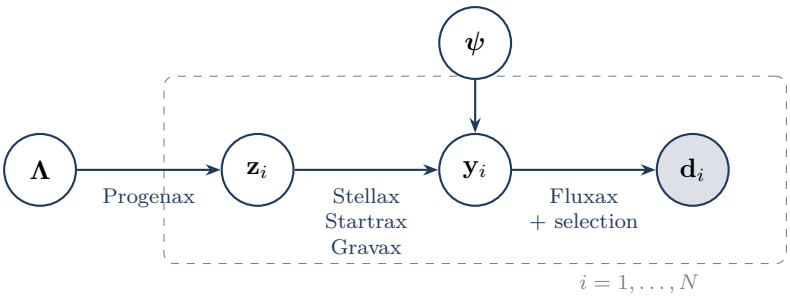
1. **Multiple chains from dispersed starts** agree: the rank-normalized split-$\hat{R}$ is below about 1.01 [7].
2. **Effective sample size** is adequate for every quantity you report, including tail quantiles, which mix more slowly than means.
3. **Divergences** (HMC trajectories whose energy error explodes) are absent. They signal regions of high curvature, typically a funnel in a hierarchical model, that the sampler cannot explore.
4. **Coordinates:** if you sample $z = \log\theta$ but your prior is written in θ, add $\log|\mathrm{d}\theta/\mathrm{d}z| = z$ to the log density.

In practice use a tested implementation, for example NUTS in NumPyro or BlackJAX, with ArviZ-style diagnostics. Write the from-scratch version once, so you know what the library is doing when it warns you.

Variational inference instead fits a tractable distribution $q_{\boldsymbol{\omega}}(\boldsymbol{\theta})$ by maximizing a lower bound on $\log p(\mathbf{d})$. It is fast and scales to many parameters, but standard choices underestimate posterior width and miss multimodality. Treat its uncertainties as approximate until checked by the methods of Section 3.7.

3.6 Hierarchical models for stellar populations

A *hierarchical* model separates what is shared from what is individual. Let $\boldsymbol{\Lambda}$ collect population hyperparameters (IMF slope, age spread, metallicity distribution, binary fraction), let $\mathbf{z}_i$ be star-specific latent variables (true initial mass, mass ratio of an unresolved companion), and let $\boldsymbol{\psi}$ be stellar or dynamical physics shared by every star.



Shaded nodes are observed; the dashed plate repeats for each star. If stars are conditionally independent given $(\boldsymbol{\Lambda}, \boldsymbol{\psi})$, the joint model factorizes as

$$p(\boldsymbol{\Lambda}, \boldsymbol{\psi}, \{\mathbf{z}_i\}, \{\mathbf{d}_i\}) = p(\boldsymbol{\Lambda})\, p(\boldsymbol{\psi}) \prod_{i=1}^{N} p(\mathbf{z}_i \mid \boldsymbol{\Lambda})\, p(\mathbf{d}_i \mid \mathbf{z}_i, \boldsymbol{\psi}). \tag{30}$$

Conditional independence is an assumption. Real clusters correlate their members through formation, dynamical evolution, mass segregation and binary interactions.

Missing physics is absorbed by the parameters you kept

If unresolved binaries are left out, the extra light they contribute is attributed to the IMF, and the fitted slope is biased. A hierarchical model with an explicit binary population makes the competition between explanations visible in the posterior, as a correlation between γ and the binary fraction.

3.6.1 Selection effects

Surveys see only stars that pass a detection criterion. Ignoring this is the most common error in population inference. Let $P_{\mathrm{det}}(\mathbf{d})$ be the probability that a star with data $\mathbf{d}$ is detected and catalogued. The likelihood for one *detected* star is the population likelihood renormalized to the detectable part of the population [8, 9]:

$$p(\mathbf{d}_i \mid \boldsymbol{\Lambda}, \boldsymbol{\psi}, \mathrm{det}) = \frac{P_{\mathrm{det}}(\mathbf{d}_i)\, p(\mathbf{d}_i \mid \boldsymbol{\Lambda}, \boldsymbol{\psi})}{\alpha(\boldsymbol{\Lambda}, \boldsymbol{\psi})}, \qquad \alpha(\boldsymbol{\Lambda}, \boldsymbol{\psi}) = \int P_{\mathrm{det}}(\mathbf{d})\, p(\mathbf{d} \mid \boldsymbol{\Lambda}, \boldsymbol{\psi})\, \mathrm{d}\mathbf{d}. \tag{31}$$

Here $p(\mathbf{d} \mid \boldsymbol{\Lambda}, \boldsymbol{\psi}) = \int p(\mathbf{d} \mid \mathbf{z}, \boldsymbol{\psi}) p(\mathbf{z} \mid \boldsymbol{\Lambda})\, \mathrm{d}\mathbf{z}$. The factor $P_{\mathrm{det}}(\mathbf{d}_i)$ does not depend on the parameters and drops out, but the *detection fraction* α does not: a steeper IMF puts more stars below the detection limit, and α is what tells the model so. α is usually computed by Monte Carlo, pushing simulated populations through the forward model and the survey's selection function. If the number of detected stars N_{obs} is itself informative, use the extended likelihood: multiply the product of Eq. (31) over detected stars by $(N_{\mathrm{exp}}\alpha)^{N_{\mathrm{obs}}} e^{-N_{\mathrm{exp}}\alpha}$, where N_{exp} is the expected total number of stars. The factor $\alpha^{N_{\mathrm{obs}}}$ cancels the per-star denominators, leaving $e^{-N_{\mathrm{exp}}\alpha}$ to carry the selection information.

3.7 Checking an inference code

A posterior can be precise and wrong. Two complementary checks catch different failures.

Posterior predictive checks [10] simulate replicated data from the posterior predictive and compare them with the observed data through a test statistic chosen to probe a suspected weakness (the colour spread on the giant branch, the number of stars above the turnoff). They test the *model*.

Simulation-based calibration (SBC) [11] tests the *inference code*, for example a sampler, an emulator-based likelihood, or a variational approximation. Repeat many times: draw $\tilde{\boldsymbol{\theta}}$ from the prior, simulate $\tilde{\mathbf{d}}$, run inference on $\tilde{\mathbf{d}}$ to get S posterior draws, and record the rank of $\tilde{\boldsymbol{\theta}}$ among them (the number of draws below it, an integer from 0 to S). If the inference is exact, the ranks are uniform, because averaging the posterior over data simulated from the prior returns the prior. Departures have recognizable shapes:

- U-shaped ranks: the posteriors are too narrow (overconfident);
- a central hump: too wide (underconfident);
- a slope: systematically biased.

ex07_sbc.py -- SBC for the conjugate model with three inference codes

```python
"""Simulation-based calibration (SBC) for the conjugate Gaussian model.

theta ~ N(0, 1),  y = 2 theta + eps,  eps ~ N(0, 1).
Exact posterior: N(2y/5, 1/5).  We test three 'inference codes':
  correct, overconfident (sd halved), and biased (mean shifted by +0.3 posterior sd).
If an inference code is calibrated, the rank of the true theta among S posterior
draws is uniform on {0, ..., S}.
"""
import jax
import jax.numpy as jnp

jax.config.update("jax_enable_x64", True)

n_sims, S = 4000, 99


def posterior(y, kind):
    mean, sd = 2.0 * y / 5.0, jnp.sqrt(0.2)
    if kind == "overconfident":
        sd = 0.5 * sd
    if kind == "biased":
        mean = mean + 0.3 * sd
    return mean, sd


def one_sim(key, kind):
    k1, k2, k3 = jax.random.split(key, 3)
    theta = jax.random.normal(k1)  # draw from the PRIOR
    y = 2.0 * theta + jax.random.normal(k2)  # simulate data
    mean, sd = posterior(y, kind)
    draws = mean + sd * jax.random.normal(k3, (S,))  # 'run inference'
    rank = jnp.sum(draws < theta)
    covered = jnp.abs(theta - mean) < sd  # nominal 68.3% central interval
    return rank, covered


keys = jax.random.split(jax.random.key(11), n_sims)
results = {}
for kind in ["correct", "overconfident", "biased"]:
    ranks, cov = jax.vmap(lambda k: one_sim(k, kind))(keys)
    counts = jnp.histogram(ranks, bins=10, range=(0, S + 1))[0]
    chi2 = jnp.sum((counts - n_sims / 10) ** 2 / (n_sims / 10))
    results[kind] = ranks
    print(f"{kind:>13}: 68% interval coverage = {cov.mean():.3f};"
          f" rank-histogram chi^2 = {chi2:6.1f} (9 dof; 95th pct = 16.9)")
```

Output (executed for these notes):

```
      correct: 68% interval coverage = 0.674; rank-histogram chi^2 =    4.8 (9 dof; 95th pct = 16.9)
 overconfident: 68% interval coverage = 0.380; rank-histogram chi^2 = 2553.5 (9 dof; 95th pct = 16.9)
       biased: 68% interval coverage = 0.664; rank-histogram chi^2 =  333.9 (9 dof; 95th pct = 16.9)
```

The biased code is the instructive one. Its 68% intervals cover the truth 66% of the time, close to nominal, yet its rank histogram fails decisively. Coverage at one level is a weak test; the full rank distribution is a strong one. SBC tests calibration averaged over the prior. It does not test whether the model describes nature; that is the job of posterior predictive checks and independent data.

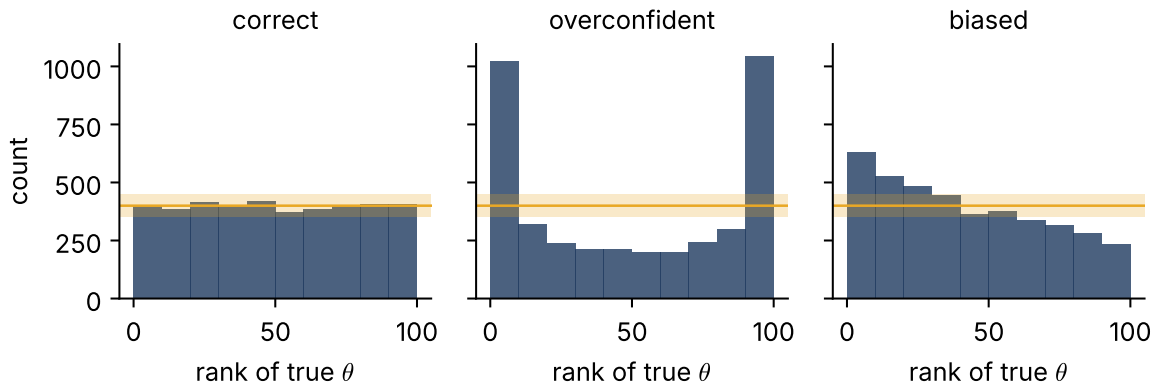


Figure 6: SBC rank histograms (4000 simulations, 99 posterior draws each). The gold band is the 99% range for a bin count under uniformity.

4 Identifiability and scientific discovery

4.1 Sensitivity is not identifiability

A parameter is *sensitive* if changing it changes a prediction. It is *identifiable* if the data can tell its values apart. These are different properties.

Worked example 4.1 (A perfect degeneracy)**.** Let a normalized observable depend on two TOY mixing coefficients through $y_1 = \theta_1 + \theta_2$. Observing $y_1 = 3$ is satisfied by the whole line $\theta_2 = 3 - \theta_1$. The observable is sensitive to each parameter, yet neither can be determined. Adding $y_2 = \theta_1 - \theta_2 = 1$ singles out $(\theta_1, \theta_2) = (2, 1)$.

Two levels of the idea should be kept apart:

Definition 4.2 (Structural and practical identifiability)**.** A model is *structurally* (globally) identifiable if $p(\mathbf{d} \mid \boldsymbol{\theta}) = p(\mathbf{d} \mid \boldsymbol{\theta}')$ for all $\mathbf{d}$ implies $\boldsymbol{\theta} = \boldsymbol{\theta}'$: with perfect, noise-free data of the planned kind, the parameters could be recovered. It is *locally* structurally identifiable at $\boldsymbol{\theta}$ if this holds in a neighbourhood. A parameter is *practically* identifiable if, with the finite, noisy data actually available, its uncertainty is small enough to answer the scientific question.

$y_1 = \theta_1 + \theta_2$ alone is structurally unidentifiable: no amount of data of that kind helps. Two nearly parallel observables, for instance $y_1 = \theta_1 + \theta_2$ and $y_2 = \theta_1 + 1.05\,\theta_2$, are structurally identifiable but may be practically unidentifiable: in principle the data separate the parameters, in practice the errors would have to be tiny. The remedies differ. Structural problems need a different *kind* of observable; practical problems may be solved by more or better data of the same kind.

For stellar evolution, luminosity alone can trade mass against metallicity, age and mixing; abundances or asteroseismic frequencies may constrain other combinations. Whether they do depends on the Jacobians and the realistic errors, which is a calculation, not an intuition.

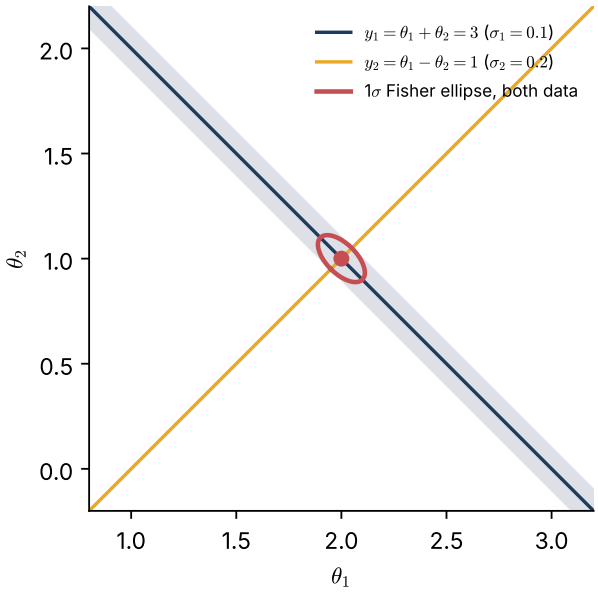


Figure 7: Each observable alone constrains a band of parameter space (shaded for y_1). Complementary observables intersect in a closed ellipse, the 1σ contour of the inverse Fisher matrix of Section 4.2.

4.2 Fisher information

Definition 4.3 (Fisher information)**.** The *score* is the gradient of the log-likelihood, $\nabla_{\boldsymbol{\theta}} \ell$. The Fisher information matrix is its covariance under the sampling distribution of the data,

$$\mathcal{I}(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{d}|\boldsymbol{\theta}}\big[\nabla \ell \, \nabla \ell^{\mathsf{T}}\big] = -\mathbb{E}_{\mathbf{d}|\boldsymbol{\theta}}\big[\nabla^2 \ell\big], \tag{32}$$

the second equality holding under standard regularity conditions (support independent of $\boldsymbol{\theta}$, differentiation under the integral).

For Gaussian data $\mathbf{d} \sim \mathcal{N}(\mathcal{G}(\boldsymbol{\theta}), \boldsymbol{\Sigma})$ with $\boldsymbol{\Sigma}$ fixed,

$$\mathcal{I}(\boldsymbol{\theta}) = \mathbf{J}(\boldsymbol{\theta})^{\mathsf{T}} \boldsymbol{\Sigma}^{-1} \mathbf{J}(\boldsymbol{\theta}), \qquad \mathbf{J} = \partial \mathcal{G} / \partial \boldsymbol{\theta}. \tag{33}$$

If the covariance depends on parameters, add $\frac{1}{2}\operatorname{tr}(\boldsymbol{\Sigma}^{-1}\partial_i \boldsymbol{\Sigma}\, \boldsymbol{\Sigma}^{-1} \partial_j \boldsymbol{\Sigma})$ to entry (i, j). Fisher matrices from *independent* data sets add, which is what makes them a planning tool.

Worked example 4.4 (The degeneracy, in matrices)**.** For $y_1 = \theta_1 + \theta_2$ with $\sigma_1 = 0.1$: $\mathbf{J}_1 = (1 \;\; 1)$ and $\mathcal{I}_1 = 100\left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right)$, with eigenvalues 200 and 0. The zero eigenvalue's eigenvector $\propto (1, -1)^{\mathsf{T}}$ is the direction the data cannot see. Adding $y_2 = \theta_1 - \theta_2$ with $\sigma_2 = 0.2$ contributes $25\left(\begin{smallmatrix} 1 & -1 \\ -1 & 1 \end{smallmatrix}\right)$, so $\mathcal{I} = \left(\begin{smallmatrix} 125 & 75 \\ 75 & 125 \end{smallmatrix}\right)$ with eigenvalues 200 and 50: both directions are now informed.

A rank-deficient Fisher matrix signals local structural non-identifiability [12]; a nearly singular one signals that some combination will be poorly constrained.

Result 4.5 (Cramér–Rao bound)**.** Under regularity conditions, any *unbiased* estimator $\hat{\boldsymbol{\theta}}$ satisfies $\operatorname{Cov}(\hat{\boldsymbol{\theta}}) \succeq \mathcal{I}(\boldsymbol{\theta})^{-1}$ (the difference is positive semi-definite).

Three cautions. The bound concerns unbiased estimators, and biased ones (including every Bayesian posterior mean with an informative prior) can beat it. It is local, so it misses multimodality, boundaries and heavy tails. And when $\mathcal{I}$ is singular, $\mathcal{I}^{-1}$ does not exist, and no unbiased estimator of the unconstrained combination exists at all. Then report the constrained combinations, or add prior information, which in the Gaussian case gives the bound $(\mathcal{I} + \boldsymbol{\Sigma}_0^{-1})^{-1}$.

4.2.1 Fisher eigenvectors depend on your units

A tempting reading of $\mathcal{I}$ is "its stiffest eigenvector is the parameter combination the data measure best". That reading depends on the units of the parameters. If $\boldsymbol{\theta}' = \mathbf{S}^{-1}\boldsymbol{\theta}$ for a diagonal rescaling $\mathbf{S}$, then $\mathcal{I}' = \mathbf{S}\mathcal{I}\mathbf{S}$, whose eigenvectors generally point elsewhere: an eigen-decomposition asks a question about Euclidean distance in parameter space, and Euclidean distance between α_{MLT} and f_{ov} has no physical meaning.

The fix is to measure information *relative to what you already knew*. Solve the generalized eigenproblem

$$\mathcal{I}\,\mathbf{v} = \lambda\,\boldsymbol{\Sigma}_0^{-1}\mathbf{v} \quad\Longleftrightarrow\quad (\mathbf{L}^{\mathsf{T}}\mathcal{I}\mathbf{L})\mathbf{u} = \lambda\mathbf{u}, \qquad \boldsymbol{\Sigma}_0 = \mathbf{L}\mathbf{L}^{\mathsf{T}}. \tag{34}$$

The eigenvalues λ_i are invariant under any linear reparameterization (the prior widths rescale with the parameters). They have a direct reading: $\lambda_i > 1$ means the data dominate the prior along that direction; $\lambda_i \ll 1$ means the data add almost nothing there.

ex05_fisher_units.py -- Fisher eigenvectors before and after changing the units of f_{ov}

```python
"""Fisher eigenvectors depend on parameter units; prior-whitened ones do not.

Toy linearized model with two hypothetical physics parameters:
  theta1 = mixing-length parameter alpha_MLT   (order 1)
  theta2 = overshoot parameter f_ov            (order 0.01)
Two observables with an assumed (toy) Jacobian and independent Gaussian errors.
"""
import jax
import jax.numpy as jnp
from jax.scipy.linalg import eigh

jax.config.update("jax_enable_x64", True)

J = jnp.array([[0.30, 12.0],    # d(obs1)/d(alpha), d(obs1)/d(f_ov)  [toy numbers]
               [0.10, 10.0]])   # d(obs2)/d(alpha), d(obs2)/d(f_ov)
Sigma_inv = jnp.diag(1.0 / jnp.array([0.02, 0.05]) ** 2)
prior_sd = jnp.array([0.2, 0.01])  # prior widths in the ORIGINAL units


def report(label, F, prior_sd):
    w, V = jnp.linalg.eigh(F)
    print(f"{label}: Fisher eigenvalues {w}, stiffest direction {V[:, -1]}")
    # prior-whitened problem: F v = lambda Sigma0^-1 v  <=>  eig of D F D, D = diag(prior_sd)
    D = jnp.diag(prior_sd)
    lam, _ = eigh(D @ F @ D)
    print(f"{'':>{len(label)}}  prior-whitened eigenvalues {lam}  (>1: data beat the prior)")


F = J.T @ Sigma_inv @ J
report("f_ov in raw units      ", F, prior_sd)

```

```
# Re-express f_ov in units of 0.01:  theta2' = theta2 / 0.01, so d/dtheta2' = 0.01 d/dtheta2
S = jnp.diag(jnp.array([1.0, 0.01]))
F_scaled = S @ F @ S
report("f_ov in units of 0.01  ", F_scaled, prior_sd / jnp.array([1.0, 0.01]))
```

Output (executed for these notes):

```
f_ov in raw units      : Fisher eigenvalues [8.09552915e+00 4.00220904e+05], stiffest direction [0.02349399
     0.99972398]
                         prior-whitened eigenvalues [ 0.26505809 48.89494191]  (>1: data beat the prior)
f_ov in units of 0.01  : Fisher eigenvalues [  1.20990284 267.79009716], stiffest direction [-0.92438626
     -0.38145779]
                         prior-whitened eigenvalues [ 0.26505809 48.89494191]  (>1: data beat the prior)
```

Changing $f_{\rm ov}$'s units rotates the stiffest eigenvector from almost pure $f_{\rm ov}$ to mostly $\alpha_{\rm MLT}$, and changes the eigenvalues by three orders of magnitude, yet nothing about the experiment changed. The prior-whitened eigenvalues (0.27 and 48.9) are identical: along one combination the data carry about 49 times the prior's precision (so the posterior standard deviation there is $\sqrt{1+48.9} \approx 7$ times smaller than the prior's), and the other is barely informed.

4.3 Beyond local: profile likelihoods

Fisher analysis linearizes. To test practical identifiability without linearizing, compute the profile likelihood $\ell_{\rm prof}(\theta_j) = \max_{\boldsymbol{\theta}_{-j}} \ell(\boldsymbol{\theta})$ on a grid of θ_j by repeated optimization [13]. A parameter is practically identifiable at 95% confidence if $-2[\ell_{\rm prof}(\theta_j) - \ell_{\rm max}]$ rises above 3.84, the 95th percentile of a χ^2 distribution with one degree of freedom, on both sides of the optimum. A profile that flattens on one side (for example toward large $f_{\rm ov}$) reveals a one-sided constraint that a Fisher ellipse would draw as symmetric.

4.4 Model discrepancy and its own identifiability problem

Every stellar model is wrong somewhere. Kennedy and O'Hagan [14] made this explicit by writing observations as

$$\mathbf{d} = \mathcal{G}(\boldsymbol{\theta}) + \boldsymbol{\delta} + \boldsymbol{\epsilon}, \tag{35}$$

with $\boldsymbol{\delta}$ an unknown, structured discrepancy (often a Gaussian process over the conditions of the observation) and $\boldsymbol{\epsilon}$ the measurement noise. Including $\boldsymbol{\delta}$ prevents a fit from forcing physical parameters to unphysical values to compensate for missing physics.

Discrepancy and parameters are confounded

A flexible $\boldsymbol{\delta}$ can mimic the effect of changing $\boldsymbol{\theta}$, so $\boldsymbol{\theta}$ and $\boldsymbol{\delta}$ are generally not jointly identifiable: the posterior on $\boldsymbol{\theta}$ becomes broad or depends strongly on the prior for $\boldsymbol{\delta}$ [15]. The remedy is physical knowledge: informative priors on the size and shape of the discrepancy (for example, "missing transport physics can change $T_{\rm eff}$ on the giant branch by up to $\sim 100\,$K but not the main-sequence luminosity"), and data whose relation to $\boldsymbol{\theta}$ the discrepancy cannot mimic.

This is why scientific claims should be stated conditionally: "within this family of stellar models, with these conventional uncertainties, the data constrain this combination of parameters to this range". Repeating the analysis with alternative prescriptions for convection, mass loss and composition is how one learns whether a conclusion survives the model's known weaknesses.

4.5 Optimal experimental design: what should we measure next?

A *design* ξ is something under our control: a photometric band, which stars to target, a cadence, the depth of a survey, where to run the next expensive simulation. A utility $\mathcal{U}(\xi)$ scores it, and the optimal design is $\xi^\star = \arg\max_\xi \mathcal{U}(\xi)$.

Definition 4.6 (Expected information gain)**.**

$$\mathcal{U}(\xi) = \mathbb{I}(\boldsymbol{\Theta}; \boldsymbol{Y} \mid \xi) = \mathbb{E}_{\boldsymbol{Y}\mid\xi}\Big[D_{\mathrm{KL}}\big(p(\boldsymbol{\theta} \mid \boldsymbol{Y}, \xi) \,\big\|\, p(\boldsymbol{\theta})\big)\Big] = \mathbb{E}_{\boldsymbol{\theta},\boldsymbol{Y}}\big[\log p(\boldsymbol{Y} \mid \boldsymbol{\theta}, \xi) - \log p(\boldsymbol{Y} \mid \xi)\big], \quad (36)$$

the expected Kullback–Leibler divergence from prior to posterior, which is the mutual information between parameters and data.

For densities, $D_{\mathrm{KL}}(p\|q) = \int p \log(p/q)$; it is non-negative, zero only when $p = q$ almost everywhere, and not symmetric. The (differential) entropy $\mathbb{H}[p] = -\int p \log p$ depends on units and coordinates, but the mutual information does not: it is invariant under invertible reparameterizations of $\boldsymbol{\theta}$ and of $\boldsymbol{Y}$. With natural logarithms it is measured in nats; divide by $\ln 2$ for bits. The last form of Eq. (36) is the one used for computation: it needs the likelihood and the evidence, but no posterior.

Result 4.7 (EIG in the linear-Gaussian model)**.** For $\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0)$ and $\boldsymbol{Y} = \mathbf{J}(\xi)\boldsymbol{\theta} + \boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}(\xi))$,

$$\mathcal{U}(\xi) = \tfrac{1}{2} \log\det\big(\mathbf{I} + \boldsymbol{\Sigma}_0\, \mathcal{I}(\xi)\big) = \tfrac{1}{2} \sum_i \log(1 + \lambda_i), \quad (37)$$

with $\mathcal{I}(\xi) = \mathbf{J}^\mathsf{T}\boldsymbol{\Sigma}^{-1}\mathbf{J}$ and λ_i the prior-whitened Fisher eigenvalues of Eq. (34). Maximizing it is Bayesian *D-optimal* design.

Equation (37) unites this section: each prior-whitened direction contributes $\frac{1}{2}\log(1 + \lambda_i)$, so a direction already well measured ($\lambda_i \gg 1$) gains only logarithmically from more of the same data, while a previously unconstrained direction gains a lot. In one dimension (the model $y = a\theta + \epsilon$ of Eq. (24)) it reduces to $\mathcal{U} = \frac{1}{2}\log(1 + a^2\tau_0^2/\sigma^2)$: with $a = 2$, $\tau_0 = 1$, $\sigma = 1$ this is $\frac{1}{2}\ln 5 \approx 0.805$ nats, and halving the noise gives $\frac{1}{2}\ln 17 \approx 1.417$ nats.

Worked example 4.8 (Precision versus complementarity)**.** Suppose we already hold $y_1 = \theta_1 + \theta_2$ with $\sigma = 0.1$ and can afford one more measurement: (a) repeat y_1 with half the error, or (b) measure the complementary $y_2 = \theta_1 - \theta_2$ with *twice* the error.

ex10_design.py -- comparing two candidate measurements by EIG

```python
"""Expected information gain (EIG) for choosing the next measurement.

Linear-Gaussian model: theta ~ N(0, Sigma0), y = J theta + noise, noise ~ N(0, Sigma).
Then EIG = 1/2 log det(I + Sigma0 J^T Sigma^-1 J)  (in nats).
We already hold y1 = theta1 + theta2 (sigma = 0.1). Which second measurement helps more?
"""
import jax
import jax.numpy as jnp

jax.config.update("jax_enable_x64", True)

Sigma0 = jnp.eye(2)


def fisher(J, sig):
    J = jnp.atleast_2d(J)
    return J.T @ jnp.diag(1.0 / jnp.asarray(sig) ** 2) @ J


def eig(F):  # EIG of all data whose total Fisher matrix is F
    return 0.5 * jnp.linalg.slogdet(jnp.eye(2) + Sigma0 @ F)[1]


F1 = fisher(jnp.array([1.0, 1.0]), [0.1])
options = {
    "(a) repeat y1, sigma = 0.05": fisher(jnp.array([1.0, 1.0]), [0.05]),
    "(b) y2 = theta1 - theta2, sigma = 0.2": fisher(jnp.array([1.0, -1.0]), [0.2]),
}
print(f"EIG of y1 alone: {eig(F1):.3f} nats")
for name, F2 in options.items():
    print(f"{name}: extra EIG = {eig(F1 + F2) - eig(F1):.3f} nats")

# Check the closed form by nested Monte Carlo in a 1-D case: y = a*theta + eps
a, s = 2.0, 1.0
k1, k2, k3 = jax.random.split(jax.random.key(5), 3)
N, M = 4000, 4000
th = jax.random.normal(k1, (N,))
y = a * th + s * jax.random.normal(k2, (N,))
th_inner = jax.random.normal(k3, (M,))
log_lik = lambda y, t: -0.5 * ((y - a * t) / s) ** 2  # constants cancel in the ratio
log_evid = jax.vmap(lambda yi: jax.scipy.special.logsumexp(log_lik(yi, th_inner))
                    - jnp.log(M))(y)
terms = log_lik(y, th) - log_evid
eig_mc, se = terms.mean(), terms.std() / jnp.sqrt(N)
print(f"1-D check: nested MC EIG = {eig_mc:.3f} +/- {se:.3f}, closed form = {0.5 * jnp.log(1 + a
    **2 / s**2):.3f}")
```

Output (executed for these notes):

```
EIG of y1 alone: 2.652 nats
(a) repeat y1, sigma = 0.05: extra EIG = 0.803 nats
(b) y2 = theta1 - theta2, sigma = 0.2: extra EIG = 1.966 nats
1-D check: nested MC EIG = 0.822 +/- 0.014, closed form = 0.805
```

The noisier but complementary measurement is worth 1.97 nats, against 0.80 nats for the more precise repeat. The last line checks Eq. (37) by nested Monte Carlo for the one-dimensional model above, estimating the evidence with an inner Monte Carlo average. The estimate, 0.822 ± 0.014, is consistent with the closed form 0.805: the difference is 1.2 standard errors. Nested estimators are

also biased: the logarithm of a noisy evidence estimate is biased low, which biases the EIG high, by an amount that shrinks as the inner sample grows (here it is far below the Monte Carlo error) [16].

In Jaxstro

Would TRGB luminosities, red-clump properties, asteroseismic frequencies, or abundance patterns constrain different combinations of uncertain Stellax physics? With a verified Stellax Jacobian for each candidate observable and realistic error budgets, Eq. (37) ranks them in minutes, before any telescope time is requested. The linear-Gaussian EIG is a first pass; near evolutionary transitions, where Eq. (12) fails, use the nested or variational estimators of [17].

4.5.1 Three different optimization questions

- **Parameter fitting:** which $\boldsymbol{\theta}$ best explains the data we have?
- **Bayesian optimization:** where should we evaluate an expensive objective next to find its minimum (for example, the stellar model that best fits a cluster)?
- **Bayesian experimental design:** which observation or simulation would teach us most about the parameters or models?

All three use surrogates, gradients and acquisition functions, but their utilities differ. Expected improvement, a Bayesian-optimization utility, is not expected information gain about stellar physics.

4.6 Surrogates, Gaussian processes, and multifidelity models

A *surrogate* (or *emulator*) $\hat{\mathcal{G}}$ is a cheap approximation of an expensive $\mathcal{G}$, *trained* on examples from $\mathcal{G}$. A Gaussian process (GP) places a prior over functions, $\mathcal{G} \sim \mathrm{GP}(m, k)$, with mean function m and kernel $k(\boldsymbol{\theta}, \boldsymbol{\theta}')$ specifying how correlated function values are at nearby inputs. Given training inputs $\mathbf{X}$ and outputs $\mathbf{f}$ (with noise variance σ_n^2), the prediction at $\boldsymbol{\theta}_*$ is Gaussian with

$$\mu_* = m(\boldsymbol{\theta}_*) + \mathbf{k}_*^\mathsf{T}(\mathbf{K} + \sigma_n^2\mathbf{I})^{-1}(\mathbf{f} - \mathbf{m}), \qquad \sigma_*^2 = k(\boldsymbol{\theta}_*, \boldsymbol{\theta}_*) - \mathbf{k}_*^\mathsf{T}(\mathbf{K} + \sigma_n^2\mathbf{I})^{-1}\mathbf{k}_*, \tag{38}$$

where $\mathbf{K}_{ij} = k(\mathbf{X}_i, \mathbf{X}_j)$ and $(\mathbf{k}_*)_i = k(\mathbf{X}_i, \boldsymbol{\theta}_*)$ [18]. For a differentiable kernel, derivative observations $\nabla\mathcal{G}(\mathbf{X}_i)$ enter by differentiating k, which is one way to put verified Stellax gradients to work: with reverse-mode AD, the gradient of a scalar output costs a small multiple of one run but carries p numbers of information.

Emulator uncertainty is not physical uncertainty

σ_* describes the surrogate's uncertainty about $\mathcal{G}$ under the kernel's assumptions, not uncertainty about the true stellar physics. If an emulator replaces Stellax inside a likelihood, add σ_*^2 to the observational variance; otherwise interpolation error masquerades as physics.

Multifidelity models combine cheap approximate and expensive accurate codes, classically as $\mathcal{G}_{\mathrm{high}}(\boldsymbol{\theta}) = a\,\mathcal{G}_{\mathrm{low}}(\boldsymbol{\theta}) + \delta(\boldsymbol{\theta})$ with a GP for δ [19]. In Jaxstro this could pair fast Startrax tracks with selected Stellax runs. Do not assume δ is small or smooth across every evolutionary phase; check it where the physics changes character. *Active learning* chooses the next training run by an acquisition function, for example expected improvement for optimization or EIG for learning; with runtimes that vary by orders of magnitude across mass, divide the utility by the cost.

5 Differentiable and trustworthy scientific computing

5.1 Automatic differentiation: JVPs and VJPs

Automatic differentiation (AD) applies the chain rule to the elementary operations a program actually executes [20]. It is exact up to floating-point rounding for the program as written. It differs from finite differences, which perturb inputs and approximate, and from symbolic differentiation, which manipulates formulas and can blow up in size.

For $\mathcal{G} : \mathbb{R}^p \to \mathbb{R}^m$ with Jacobian $\mathbf{J}$ there are two primitive operations:

- a **Jacobian–vector product** (JVP) $\mathbf{Jv}$: how all outputs change along one input direction $\mathbf{v}$. Forward mode computes it by carrying a tangent alongside each value through the program ("dual numbers", $x + \epsilon\dot{x}$ with $\epsilon^2 = 0$).
- a **vector–Jacobian product** (VJP) $\mathbf{u}^\mathsf{T}\mathbf{J}$: how one weighted combination of outputs depends on all inputs. Reverse mode computes it by running the program forward, storing intermediates, then propagating sensitivities backward.

Task	Forward mode	Reverse mode
gradient of a scalar loss ($m = 1$)	p passes	1 pass (a few times the cost of $\mathcal{G}$)
full Jacobian	p passes	m passes
memory	like $\mathcal{G}$	grows with the length of the computation

A scalar loss of many parameters favours reverse mode; a Jacobian with few inputs and many outputs (5 physics parameters, 2000 points on a track) favours forward mode. Reverse mode's memory cost is the practical obstacle for long stellar evolutions, mitigated by checkpointing (recomputing intermediates instead of storing them) and by the implicit methods of Section 5.2. In JAX: `jax.jvp`, `jax.vjp`, `jax.grad`, `jax.jacfwd`, `jax.jacrev`, `jax.hessian`, `jax.linearize`, and `jax.vmap` to batch any of them.

Key idea

AD returns the derivative of *the program*. Whether that equals the derivative of *the physics* is a separate question, answered by Sections 5.3 and 5.4.

5.2 Differentiating through an implicit solve: the adjoint method

Stellar structure defines the state implicitly, $\mathbf{R}(\mathbf{s}, \boldsymbol{\theta}) = \mathbf{0}$ (Eq. (15)). Differentiating the identity $\mathbf{R}(\mathbf{s}(\boldsymbol{\theta}), \boldsymbol{\theta}) = \mathbf{0}$ gives

$$\mathbf{R}_s \frac{\mathrm{d}\mathbf{s}}{\mathrm{d}\boldsymbol{\theta}} + \mathbf{R}_\theta = \mathbf{0} \quad \Longrightarrow \quad \frac{\mathrm{d}\mathbf{s}}{\mathrm{d}\boldsymbol{\theta}} = -\mathbf{R}_s^{-1}\mathbf{R}_\theta, \tag{39}$$

with $\mathbf{R}_s = \partial\mathbf{R}/\partial\mathbf{s}$ (the Jacobian the Newton solver already factors) and $\mathbf{R}_\theta = \partial\mathbf{R}/\partial\boldsymbol{\theta}$. This is the *implicit function theorem*: no derivative of the Newton iterations themselves is needed, only of the residual at the solution. Forming $\mathrm{d}\mathbf{s}/\mathrm{d}\boldsymbol{\theta}$ costs p linear solves.

Usually we want the gradient of a scalar $\mathcal{L}(\mathbf{s}(\boldsymbol{\theta}), \boldsymbol{\theta})$. By the chain rule,

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{\theta}} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} + \frac{\partial \mathcal{L}}{\partial \mathbf{s}} \frac{\mathrm{d}\mathbf{s}}{\mathrm{d}\boldsymbol{\theta}} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} - \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{s}} \mathbf{R}_s^{-1}}_{\boldsymbol{\lambda}^\mathsf{T}} \mathbf{R}_\theta.$$

Grouping the product the other way defines the *adjoint* vector $\boldsymbol{\lambda}$:

$$\mathbf{R}_s^\mathsf{T} \boldsymbol{\lambda} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{s}}\right)^\mathsf{T}, \qquad \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{\theta}} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} - \boldsymbol{\lambda}^\mathsf{T} \mathbf{R}_\theta. \tag{40}$$

One transposed linear solve gives the whole gradient, however many parameters there are. The adjoint method is reverse-mode AD applied at the level of the solver, and it is how gradients of PDE- and ODE-constrained problems are computed in practice.

ex09_implicit_adjoint.py -- four ways to differentiate through a nonlinear solve

```python
"""Gradients through an implicit (nonlinear) solve: unrolled AD, forward
sensitivities, the adjoint method, and jax.lax.custom_root.

Toy 'structure' problem: find s in R^n with  R(s, theta) = A s + s**3 - theta = 0,
where A is a symmetric positive-definite discrete Laplacian (so the solution is unique).
Scalar objective: Lfun(s) = 1/2 ||s - s_obs||^2.
"""
import jax
import jax.numpy as jnp

jax.config.update("jax_enable_x64", True)

n = 50
A = (jnp.diag(2.0 * jnp.ones(n)) - jnp.diag(jnp.ones(n - 1), 1)
     - jnp.diag(jnp.ones(n - 1), -1)) * n**2 / 100.0
x = jnp.linspace(0, 1, n)
theta = 1.0 + jnp.sin(2 * jnp.pi * x)  # 'physics parameters' (here: a source profile)
s_obs = 0.5 * jnp.ones(n)


def R(s, theta):
    return A @ s + s**3 - theta


def newton_solve(theta, iters=30):
    s = jnp.zeros(n)
    for _ in range(iters):  # Python loop: jax.grad will unroll and differentiate it
        s = s - jnp.linalg.solve(jax.jacfwd(R)(s, theta), R(s, theta))
    return s


def objective(s):
    return 0.5 * jnp.sum((s - s_obs) ** 2)


# (1) Differentiate through every Newton iteration (memory grows with iterations)
g_unrolled = jax.grad(lambda th: objective(newton_solve(th)))(theta)

# At the solution, form the Jacobians of the residual
s_star = newton_solve(theta)
R_s = jax.jacfwd(R, argnums=0)(s_star, theta)  # n x n
R_th = jax.jacfwd(R, argnums=1)(s_star, theta)  # n x n (here: -I)
```

```
dL_ds = jax.grad(objective)(s_star)

# (2) Forward sensitivities: ds/dtheta = -R_s^{-1} R_th  -> n linear solves
ds_dth = -jnp.linalg.solve(R_s, R_th)
g_forward = ds_dth.T @ dL_ds

# (3) Adjoint: ONE transposed solve, R_s^T lam = dL/ds, then g = -lam^T R_th
lam = jnp.linalg.solve(R_s.T, dL_ds)
g_adjoint = -R_th.T @ lam


# (4) Let JAX apply the implicit function theorem for us
def solve_implicit(theta):
    newton = lambda f, s0: newton_solve(theta)  # the forward solve itself
    lin = lambda g, y: jnp.linalg.solve(jax.jacfwd(g)(y), y)  # tangent solve
    return jax.lax.custom_root(lambda s: R(s, theta), jnp.zeros(n), newton, lin)


g_custom = jax.grad(lambda th: objective(solve_implicit(th)))(theta)

print(f"residual norm at solution : {jnp.linalg.norm(R(s_star, theta)):.1e}")
for name, g in [("forward sens.", g_forward), ("adjoint", g_adjoint),
                ("custom_root", g_custom)]:
    print(f"max |g_{name} - g_unrolled| = {jnp.max(jnp.abs(g - g_unrolled)):.1e}")
```

Output (executed for these notes):

```
residual norm at solution : 3.4e-14
max |g_forward sens. - g_unrolled| = 8.3e-17
max |g_adjoint - g_unrolled| = 5.6e-17
max |g_custom_root - g_unrolled| = 8.3e-17
```

All four agree to rounding error. They differ in cost: unrolling stores every Newton iterate; forward sensitivities need p solves; the adjoint needs one; `jax.lax.custom_root` applies the implicit function theorem automatically. Libraries such as Optimistix (implicit adjoints for root finds and minimizations) and Diffrax (adjoints for ODE solves) package the same idea; see [21] for a general treatment of implicit differentiation in JAX.

A singular $\mathbf{R}_s$ is information

Equations (39) and (40) need $\mathbf{R}_s$ to be invertible. Near a fold or bifurcation of the solution family (for instance, where two structure solutions merge) $\mathbf{R}_s$ becomes singular and sensitivities diverge. That is the mathematics reporting a qualitative change in the physics, not a numerical nuisance to be regularized away.

5.3 Verifying a derivative

Every gradient used for inference should be checked against an independent method at representative points, especially near regime changes.

Finite differences and their error budget. The forward difference $[f(x+h) - f(x)]/h$ has truncation error $\approx \frac{1}{2}|f''|h$ and rounding error $\approx \epsilon_{\text{mach}}|f|/h$; their sum is smallest at $h \sim \sqrt{\epsilon_{\text{mach}}} \approx 10^{-8}$, giving relative accuracy about 10^{-8}. The central difference $[f(x+h) - f(x-h)]/2h$ has

truncation error $O(h^2)$, so the optimum moves to $h \sim \epsilon_{\text{mach}}^{1/3} \approx 6 \times 10^{-6}$ and the accuracy to about 10^{-10}. Smaller is not better: below the optimum, rounding wins.

The complex step. For a real-analytic f implemented with complex-safe operations, $f(x+ih) = f(x)+ihf'(x)+O(h^2)$, so $f'(x) \approx \operatorname{Im} f(x+ih)/h$ involves no subtraction and is accurate to machine precision for any tiny h, even 10^{-30} [22]. It fails for non-analytic operations (`abs`, comparisons, `max`).

In many dimensions, check directional derivatives: for random $\mathbf{v}$, compare the JVP $\mathbf{Jv}$ with $[\mathcal{G}(\boldsymbol{\theta}+h\mathbf{v}) - \mathcal{G}(\boldsymbol{\theta}-h\mathbf{v})]/2h$, and check reverse against forward mode with the identity $\mathbf{u}^\mathsf{T}(\mathbf{Jv}) = (\mathbf{u}^\mathsf{T}\mathbf{J})\mathbf{v}$. `jax.test_util.check_grads` automates these comparisons.

ex08_gradient_checks.py -- derivative checks, and a confidently wrong gradient

```python
"""Verifying derivatives: finite differences, complex step, check_grads,
and a case where AD returns a confidently wrong answer (an event threshold).
"""
import numpy as np
import jax
import jax.numpy as jnp
from jax.test_util import check_grads

jax.config.update("jax_enable_x64", True)


# --- Part A: a smooth function (dimensionless Planck-like occupation number) ---
def f(T):
    return 1.0 / (jnp.exp(1.0 / T) - 1.0)


def df_exact(T):
    e = np.exp(1.0 / T)
    return e / (T**2 * (e - 1.0) ** 2)


T0 = 0.3
print("Part A: d/dT [1/(exp(1/T)-1)] at T = 0.3")
print(f"  exact      = {df_exact(T0):.15e}")
print(f"  jax.grad   = {jax.grad(f)(T0):.15e}")
for h in [1e-2, 1e-5, 1e-8, 1e-11]:
    fwd = (f(T0 + h) - f(T0)) / h
    ctr = (f(T0 + h) - f(T0 - h)) / (2 * h)
    print(f"  h = {h:.0e}: forward rel.err = {abs(fwd / df_exact(T0) - 1):.1e},"
          f" central rel.err = {abs(ctr / df_exact(T0) - 1):.1e}")
f_np = lambda T: 1.0 / (np.exp(1.0 / T) - 1.0)  # numpy version accepts complex input
cs = np.imag(f_np(T0 + 1e-30j)) / 1e-30  # complex step: no subtractive cancellation
print(f"  complex step (h = 1e-30) rel.err = {abs(cs / df_exact(T0) - 1):.1e}")
check_grads(f, (T0,), order=2, modes=["fwd", "rev"])  # raises if AD disagrees with FD
print("  check_grads: passed (1st and 2nd order, forward and reverse mode)")


# --- Part B: an event threshold inside a fixed-step time loop (toy) ---
# L(t) = exp(theta * t); a wind switches on when L > L_c and removes mass at a
# fixed rate. Mass lost by time t_end:  Delta M = rate * (t_end - t_s),
# with event time t_s = ln(L_c) / theta.
L_c, rate, t_end, n_steps = 10.0, 1.0, 5.0, 2000
```

```python
dt = t_end / n_steps


def mass_lost_loop(theta):
    def body(i, dm):
        t = i * dt
        wind_on = jnp.exp(theta * t) > L_c  # boolean: no derivative flows through it
        return dm + jnp.where(wind_on, rate * dt, 0.0)
    return jax.lax.fori_loop(0, n_steps, body, 0.0)


def mass_lost_event(theta):  # locate the event, then differentiate through t_s(theta)
    t_s = jnp.log(L_c) / theta
    return rate * (t_end - t_s)


th = 1.0
exact = rate * jnp.log(L_c) / th**2
h = 1e-2  # much wider than one stair (theta changes by ~dt*theta^2/ln(L_c) ~ 1e-3 per stair)
fd = (mass_lost_loop(th + h) - mass_lost_loop(th - h)) / (2 * h)
print("\nPart B: d(Delta M)/d(theta) at theta = 1")
print(f"  exact (continuum)       = {exact:.4f}")
print(f"  jax.grad through loop   = {jax.grad(mass_lost_loop)(th):.4f}   <-- wrong")
print(f"  central FD, h = 1e-2    = {fd:.4f}")
print(f"  jax.grad, event-aware   = {jax.grad(mass_lost_event)(th):.4f}")
```

Output (executed for these notes):

```
Part A: d/dT [1/(exp(1/T)-1)] at T = 0.3
  exact      = 4.262471213033907e-01
  jax.grad   = 4.262471213033907e-01
  h = 1e-02: forward rel.err = 2.6e-02, central rel.err = 3.4e-04
  h = 1e-05: forward rel.err = 2.6e-05, central rel.err = 3.3e-10
  h = 1e-08: forward rel.err = 2.5e-08, central rel.err = 1.7e-09
  h = 1e-11: forward rel.err = 1.4e-06, central rel.err = 5.5e-07
  complex step (h = 1e-30) rel.err = 0.0e+00
  check_grads: passed (1st and 2nd order, forward and reverse mode)

Part B: d(Delta M)/d(theta) at theta = 1
  exact (continuum)       = 2.3026
  jax.grad through loop   = 0.0000   <-- wrong
  central FD, h = 1e-2    = 2.3750
  jax.grad, event-aware   = 2.3026
```

Single precision

In 32-bit arithmetic $\epsilon_{\text{mach}} \approx 1.2 \times 10^{-7}$, so even the best central difference is accurate only to about 10^{-5}, and an ill-conditioned linear solve can lose every digit. Do derivative checks, Newton solves and Fisher matrices in 64-bit.

5.4 When AD is confidently wrong

Events inside a fixed-step solver. Part B of Listing 9 models a wind that switches on when $L(t) = e^{\theta t}$ crosses a threshold L_c. The continuum mass loss $\Delta M = \dot{M}\,(t_{\text{end}} - t_s)$ depends on θ through the event time $t_s = \ln L_c/\theta$, so $\mathrm{d}\Delta M/\mathrm{d}\theta = \dot{M} \ln L_c/\theta^2 = 2.30$. The fixed-step loop evaluates $\texttt{where}(L > L_c, \dot{M}\,\Delta t, 0)$ at each step. For fixed θ the boolean has no derivative, and the rate does not depend on θ, so AD returns exactly **zero**. It is the correct derivative of the program, which is a staircase in θ (Fig. 9); it is not the derivative of the physics. A finite difference with h

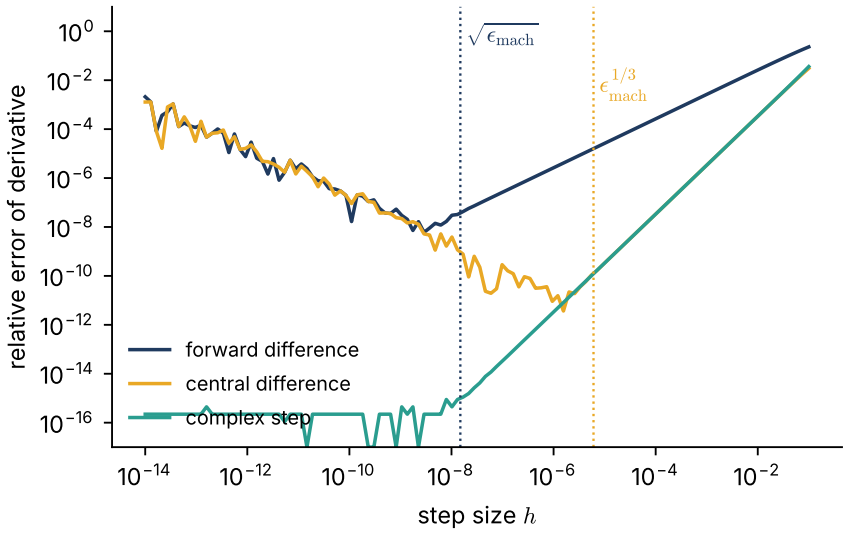


Figure 8: Relative error of three numerical derivatives of $1/(e^{1/T}-1)$ at $T=0.3$ against step size. Each finite difference has a V-shaped error with its minimum at the predicted step (dotted lines); the complex step is exact to rounding for every small h.

large compared with the stair width recovers the slope approximately (2.38; the residual error is the quantization of the staircase). Locating the event and differentiating $t_s(\theta)$ restores the exact answer.

This is the generic failure for hybrid systems: phase transitions in stellar evolution, close encounters and collisions in N-body integration, any adaptive scheme whose control flow depends on the state. The derivative through a fixed execution path omits how the event time, the jump at the event, and any change of topology depend on the parameters. Remedies are event location with implicit differentiation of the event time, saltation matrices that propagate sensitivities across a jump, and solver-level event handling (for example in Diffrax).

In Jaxstro

Startrax phase switches, Stellax helium ignition and convective-boundary changes, and Gravax close encounters are all events. A gradient-based fit or HMC run that crosses one with a path-wise derivative will see a flat or misleading landscape there. Check gradients by finite differences with h larger than the solver's event resolution on both sides of each known transition.

Discontinuities inside an expectation are the second failure, treated next.

5.5 Randomness and differentiability

To differentiate an expectation $\mathbb{E}_{X\sim p_{\boldsymbol{\theta}}}[f(X)]$ with respect to distribution parameters, write the random variable as a smooth function of parameter-free noise. For a Gaussian, $X=\mu+\sigma\epsilon$ with $\epsilon\sim\mathcal{N}(0,1)$, so

$$\frac{\partial}{\partial\mu}\mathbb{E}_{\epsilon}\big[f(\mu+\sigma\epsilon)\big]=\mathbb{E}_{\epsilon}\big[f'(\mu+\sigma\epsilon)\big] \tag{41}$$

whenever differentiation and expectation can be exchanged, which holds when f is continuous and differentiable almost everywhere with a suitably bounded derivative. This is the *reparameterization*

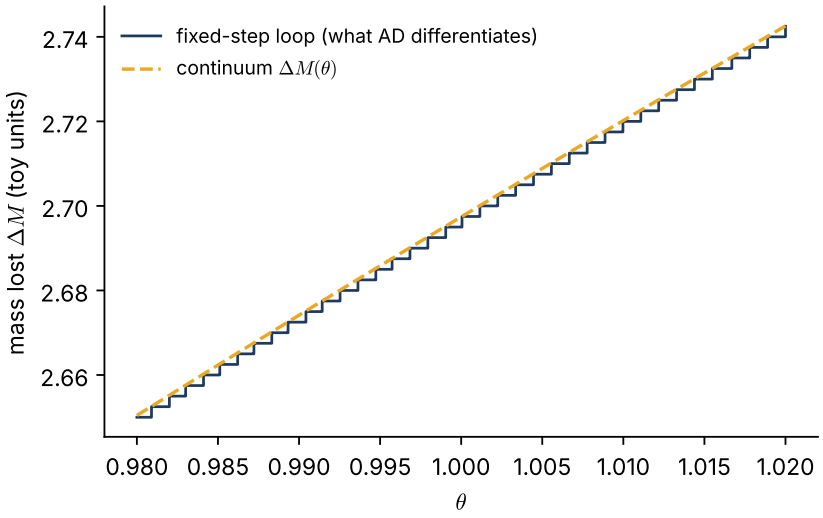


Figure 9: The program AD differentiates is a staircase whose slope is zero almost everywhere; the physical quantity is the smooth dashed line.

or *pathwise* estimator [23]. In JAX, fix the random key (or the draws) and call `jax.grad` on the sample average, as in Listing 1. Holding draws fixed while comparing nearby parameters is also called using *common random numbers*; it removes sampling noise from the comparison.

In Jaxstro: differentiating with respect to the IMF slope

Inverse-transform sampling (Eq. (6)) is a reparameterization: $M = F^{-1}(U;\gamma)$ is smooth in γ for fixed U. So gradients of population statistics with respect to γ come from `jax.grad` through a Progenax-style sampler with fixed uniforms, as long as the statistic is a smooth function of the masses.

ex02_imf_pathwise.py -- pathwise IMF gradients: a smooth statistic and an indicator

```python
"""Inverse-CDF IMF sampling as a reparameterization, and where pathwise gradients fail.

p(m | gamma) proportional to m^(-gamma) on [m_lo, m_hi]  (single power law, gamma != 1).
"""
import jax
import jax.numpy as jnp

jax.config.update("jax_enable_x64", True)

m_lo, m_hi, m_crit = 0.5, 100.0, 8.0  # Msun; m_crit = toy core-collapse threshold


def inv_cdf(u, gamma):  # M = F^{-1}(U; gamma): differentiable in gamma for fixed u
    k = 1.0 - gamma
    return (m_lo**k + u * (m_hi**k - m_lo**k)) ** (1.0 / k)


def mean_mass_exact(gamma):
    k = 1.0 - gamma
    return (k / (k + 1.0)) * (m_hi**(k + 1) - m_lo**(k + 1)) / (m_hi**k - m_lo**k)


def frac_above_exact(gamma):  # P(M > m_crit)
    k = 1.0 - gamma
    return (m_hi**k - m_crit**k) / (m_hi**k - m_lo**k)


u = jax.random.uniform(jax.random.key(1), (400_000,))  # fixed draws
gamma = 2.35  # Salpeter-like slope


def mean_mass_mc(g):
    return jnp.mean(inv_cdf(u, g))


def frac_above_mc(g):  # an indicator: piecewise constant in gamma for fixed u
    return jnp.mean((inv_cdf(u, g) > m_crit).astype(jnp.float64))


def frac_above_smooth(g, width=0.25):  # sigmoid relaxation: biased but differentiable
    return jnp.mean(jax.nn.sigmoid((inv_cdf(u, g) - m_crit) / width))


if __name__ == "__main__":
    print(f"E[M]        MC = {mean_mass_mc(gamma):.4f}  exact = {mean_mass_exact(gamma):.4f}")
    print(f"dE[M]/dg    pathwise = {jax.grad(mean_mass_mc)(gamma):+.4f}"
          f"  exact = {jax.grad(mean_mass_exact)(gamma):+.4f}")
    print(f"P(M>8)      MC = {frac_above_mc(gamma):.5f}  exact = {frac_above_exact(gamma):.5f}")
    print(f"dP/dg       pathwise = {jax.grad(frac_above_mc)(gamma):+.5f}"
          f"  exact = {jax.grad(frac_above_exact)(gamma):+.5f}")
    print(f"dP/dg       sigmoid-relaxed = {jax.grad(frac_above_smooth)(gamma):+.5f}")
```

Output (executed for these notes):

```
E[M]        MC = 1.6308  exact = 1.6279
dE[M]/dg    pathwise = -1.8547  exact = -1.8513
P(M>8)      MC = 0.02280  exact = 0.02292
dP/dg       pathwise = +0.00000  exact = -0.06166
dP/dg       sigmoid-relaxed = -0.06136
```

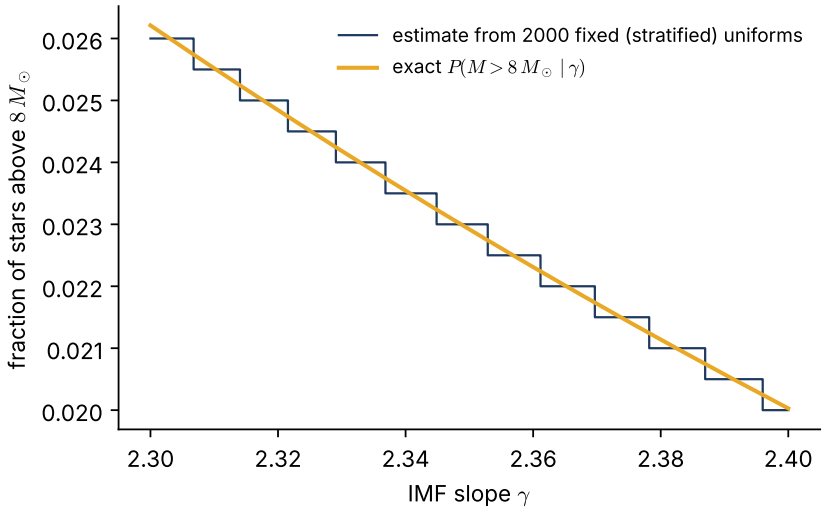


Figure 10: With the uniforms held fixed, the estimated fraction of stars above $8\,M_\odot$ is piecewise constant in γ, so its pathwise derivative is zero almost everywhere. Stratified uniforms are used here to remove sampling noise and show the steps.

For the mean mass, the pathwise gradient agrees with the exact one to Monte Carlo accuracy. For the fraction of stars above $8\,M_\odot$, it returns **zero**, while the true derivative is -0.062: a steeper IMF makes fewer massive stars. For fixed uniforms the indicator $\mathbb{1}[F^{-1}(u;\gamma) > 8]$ is a step function of γ (Fig. 10). The missing contribution is the Leibniz boundary term, the rate at which probability mass crosses $m = 8\,M_\odot$ as γ changes, and Eq. (41) does not apply because f is discontinuous.

Remedies, in order of preference:

1. **Integrate the discontinuity analytically** where you can: here, use the CDF, $P(M > 8) = 1 - F_M(8;\gamma)$, which is smooth in γ.
2. **Score-function (likelihood-ratio) estimator:** $\nabla_{\boldsymbol{\theta}} \mathbb{E}_{p_{\boldsymbol{\theta}}}[f(X)] = \mathbb{E}_{p_{\boldsymbol{\theta}}}[f(X) \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(X)]$ when f does not itself depend on $\boldsymbol{\theta}$. Unbiased for discontinuous f and discrete X, but often high-variance; subtract a baseline to reduce it.
3. **Smooth relaxation:** replace the step by a sigmoid of width w. Differentiable but *biased* by an amount that depends on w. For $w = 0.25\,M_\odot$ the relaxed derivative, computed by quadrature, is -0.0619 against the exact -0.0617, a 0.3% bias that is smaller than the Monte Carlo error of the listing's estimate (-0.0614 ± 0.0012). Report w and test the sensitivity of conclusions to it.

Remnant type, detection above a flux limit, and membership in a colour cut are all indicators. Each needs one of these treatments before its gradient means anything.

5.6 Parameters, hyperparameters, and training jargon

A *physical parameter* controls the astrophysical model (a mixing efficiency). A *model weight* is a fitted coefficient of a surrogate (a neural-network weight). A *hyperparameter* controls a fitting procedure or a level of a hierarchical model (a learning rate, a GP length scale, a prior width). The same quantity can play different roles at different levels of a hierarchy, so say which you mean.

A *batch* is a set of examples processed together; a *mini-batch* is a subset used for one update;

stochastic gradient descent estimates the full gradient from a mini-batch. *Momentum* accumulates past gradients. *Gradient clipping* caps gradient norms to stabilize training.

The *training set* updates weights. The *validation set* chooses architectures, regularization and when to stop (*early stopping*). The *test set* is inspected once, after every choice is frozen. A *metric* is what you report (maximum relative luminosity error, energy drift); it need not be the loss you trained on. *Overfitting* learns quirks of the training data; *underfitting* misses real structure; *generalization* is performance on new cases; *distribution shift* is a mismatch between training and deployment inputs; *out-of-distribution* inputs lie outside the training support, where emulators fail without warning.

Leakage between neighbouring timesteps

Splitting timesteps of the same tracks at random between training and test sets puts near-duplicates of every test state in the training set. The test error then measures interpolation along tracks the network has already seen. Hold out whole tracks, and where the science requires it, whole regions of mass and metallicity or whole evolutionary phases.

Calibration has two meanings. *Physical* calibration fits a model to reference data. *Probabilistic* calibration asks whether stated probabilities match frequencies over many cases, which is what Section 3.7 tests. Say which one you mean.

6 Two research workflows

The tools of Sections 1 to 5 become research only when combined in the right order. Two templates follow. Each step names the section that supplies its tools and the failure it guards against.

6.1 Workflow 1: how does uncertain opacity affect the TRGB?

Rather than one scalar opacity multiplier, parameterize a smooth, correlated perturbation of the opacity table:

$$\log \kappa(T, \rho, \boldsymbol{X}) = \log \kappa_0(T, \rho, \boldsymbol{X}) + \sum_{k=1}^{K} a_k \, b_k(T, \rho, \boldsymbol{X}), \tag{42}$$

with reference opacity κ_0 (here κ is the opacity, not a condition number), basis functions b_k over thermodynamic conditions, and uncertain coefficients a_k. Exponentiation keeps κ positive. The prior covariance of $\boldsymbol{a}$ should come from comparisons between independent opacity calculations or published error estimates; a correlated Gaussian is a modelling choice, not a fact about atomic physics.

1. **Targets.** TRGB luminosity, helium-core mass at the flash, and surface abundance signatures, each with units and the mass and metallicity range in which it is defined.
2. **Probability model** (Section 3). Priors for $\boldsymbol{a}$ and the other microphysics, including correlations; a likelihood for the data, including systematic floors (Section 1.7).
3. **Verify the forward model.** Convergence of Stellax in mesh and timestep for each regime, including the approach to the helium flash.
4. **Verify the derivatives** (Sections 5.3 and 5.4). Directional finite-difference checks at representative points, on both sides of every event.

5. **Propagate** (Section 1.6). $\mathbf{J}\boldsymbol{\Sigma}_0\mathbf{J}^\mathsf{T}$ first; then an ensemble near the helium flash, where linearization is least trustworthy.
6. **Identifiability** (Section 4). Prior-whitened Fisher eigenvalues; profile likelihoods for the weakly constrained directions; sampling if the posterior is non-Gaussian.
7. **Design** (Section 4.5). Rank candidate observables by EIG with realistic errors and costs.
8. **Connect to observations.** Forward-model instrument response, extinction, distances and selection (Section 3.6.1) rather than comparing theoretical outputs to catalogue values directly.
9. **Check the inference** (Section 3.7). SBC on simulated data before touching real data; posterior predictive checks after.
10. **Alternative physics** (Section 4.4). Repeat with different convection, mass-loss and composition prescriptions.
11. **Qualified claims.** What is constrained within the tested model family, what remains degenerate, and which future data would break the degeneracy.

The result is more than an error bar on one track: a map of which questions about opacity the data can answer.

6.2 Workflow 2: inferring an IMF slope from a cluster

A minimal generative model is

$$M_i \sim p(M \mid \gamma), \qquad \mathbf{y}_i = \mathcal{G}_{\text{evol}}(M_i, \boldsymbol{\psi}), \qquad \mathbf{d}_i \sim p(\mathbf{d} \mid \mathbf{y}_i, \boldsymbol{\zeta}) \text{ if detected},$$

with shared physics $\boldsymbol{\psi}$ and observing conditions $\boldsymbol{\zeta}$. Extend it with unresolved binaries and with dynamical evolution where relevant: mass segregation and evaporation make the observed stars a biased sample of the initial population.

1. Write the selection-corrected likelihood (Eq. (31)); compute $\alpha(\boldsymbol{\Lambda})$ by pushing simulated populations through the full forward model.
2. Treat a binary fraction and mass-ratio distribution as part of $\boldsymbol{\Lambda}$; expect a posterior correlation with γ.
3. Use gradients with respect to γ through inverse-CDF sampling (Section 5.5). Detection thresholds and remnant boundaries are indicators, so treat each with one of the remedies listed there before trusting its gradient.
4. Sample the joint posterior with HMC (Section 3.5); hierarchical models often need a non-centred parameterization to avoid divergences.
5. Run SBC with populations simulated from the prior before analysing the real cluster.

An optimizer gives a fast first fit. A credible conclusion needs the joint posterior, the selection model, and checks on independent data.

6.3 A recommended progression for a research student

1. A controlled, differentiable TOY model with known answers. Verify every derivative and every propagated uncertainty analytically.

2. A narrow, verified Stellax regime (for example main-sequence models in a small mass range).
3. Calibrated physical uncertainties, with priors you can defend.
4. Local sensitivity and identifiability: Fisher, prior-whitened eigenvalues, profiles.
5. SBC on simulated data.
6. Only then, joint posterior inference on real data and information-optimal design.

This order keeps impressive-looking optimization output from outrunning the numerical and physical evidence behind it.

A Exercises

Exercises are tagged **[P]** for pencil and paper, **[C]** for coding (start from the listings), and **[R]** for research judgement, where there is no single right answer but there are clearly wrong ones. None of them repeats a worked example. Solutions are in Appendix B; attempt each before reading its solution.

Probability and propagation

1. **[P] Sets of minimizers.** Let $\mathcal{L}(\theta) = \theta^4 - 2\theta^2$. Find $\min_\theta \mathcal{L}$, the set $\arg\min_\theta \mathcal{L}$, and every stationary point with its type. What would a gradient-based library return if started at $\theta_0 = 0$? At $\theta_0 = 10^{-6}$? What does this suggest about reporting a single best fit when a model has a symmetry?

2. **[P] Mean, median, mode.** For the IMF $p(m) = Cm^{-2}$ on $[1, 10]\,M_\odot$: find C, $P(M > 5\,M_\odot)$, the median and the mean. Order the mode, median and mean, and explain the ordering.

3. **[P] Log-uniform survives power laws.** Masses are log-uniform on $[m_{\mathrm{lo}}, m_{\mathrm{hi}}]$ ($\gamma = 1$) and $L = M^{3.5}$. Derive $p_L(L)$ with Eq. (10). What general property of log-uniform distributions does the result illustrate?

4. **[P] When the expectation does not exist.** A TOY main-sequence lifetime scales as $t \propto M^{-2.5}$. Let $X = M/M_\odot \sim \mathcal{N}(1, 0.1^2)$. (a) Estimate $\mathbb{E}[t]$ and $\mathrm{sd}(t)$ by the first-order delta method. (b) Add the second-order correction to the mean. (c) Is the exact $\mathbb{E}[X^{-2.5}]$ finite? What would a Monte Carlo estimate with 10^6 draws report, and why is that misleading? (d) Repeat (b) for a lognormal input, $\log X \sim \mathcal{N}(0, 0.1^2)$, and compare with the exact value.

5. **[P] Signs of correlation.** With $\sigma_1 = 2$, $\sigma_2 = 3$, $\rho_{12} = -0.5$, compute $\mathrm{Var}(X_1 + X_2)$ and $\mathrm{Var}(X_1 - X_2)$. Give a stellar-physics situation where two error sources are anticorrelated, and say which combination of them is then better known.

6. **[P] How many stars until the floor?** With a shared offset $\sigma_\theta = 0.1$ and per-star scatter $\sigma_\epsilon = 0.2$, how many stars bring $\mathrm{sd}(\bar{Y})$ within 10% of its $N \to \infty$ limit? What, besides more stars, could reduce the limit?

Optimization

7. **[P] Stability on an anisotropic bowl.** For $\mathcal{L} = \frac{1}{2}(\theta_1^2 + 50\,\theta_2^2)$: (a) the largest stable fixed step; (b) the best fixed step and its contraction factor; (c) roughly how many iterations reduce the

error by 10^6; (d) how many Newton steps; (e) a change of variables that makes gradient descent converge in one step.

8. **[P] Why the Gauss–Newton matrix is exact here.** In Listing 3, the Gauss–Newton matrix equals the full Hessian in the (a, a) and (a, b) entries at the optimum, and differs only in (b, b). Prove this. (Hint: compute $\partial^2 r_i/\partial a^2$ and $\partial^2 r_i/\partial a\,\partial b$ for $r_i \propto e^a e^{-t_i/e^b} - d_i$, and use the stationarity condition.)

9. **[C] A noise model that matters.** Modify Listing 3 to fit Poisson counts $n_i \sim \text{Poisson}(\mu_i)$ with $\mu_i = e^a e^{-t_i/e^b}$ and $e^a = 8$, using the Poisson negative log-likelihood. Fit the same data with χ^2 and $\sigma_i = \sqrt{n_i}$. Repeat over 500 simulated data sets and compare the bias of $\hat{a}$ under the two losses. Explain the sign of the χ^2 bias.

Inference

10. **[P] Two measurements, one posterior.** Prior $\theta \sim \mathcal{N}(0, 1)$; data $y_1 = \theta + \epsilon_1$ ($\sigma_1 = 1$, observed 0.8) and $y_2 = 3\theta + \epsilon_2$ ($\sigma_2 = 2$, observed 1.5). Find the posterior. Show that updating on y_1 and then on y_2 gives the same answer as updating on both at once.

11. **[P] Moving MAP, fixed median.** Let $p(\theta \mid \mathbf{d}) \propto \theta^{k-1} e^{-\theta/s}$ on $\theta > 0$. Find the MAP in θ and in $z = \log\theta$. For $k = 3$, $s = 1$, evaluate both. Which summary is invariant, and why?

12. **[C] Tuning HMC, and a familiar stability limit.** In Listing 4, run HMC with step sizes $\varepsilon \in \{0.05, 0.15, 0.25, 0.30, 0.45\}$ at 20 leapfrog steps. Record acceptance rate and ESS. Explain the collapse you observe using the eigenvalues of the target's precision matrix, and connect it to the step-size limit of Section 2.4. Why is the smallest step inefficient although it is almost always accepted?

13. **[C] SBC finds what coverage misses.** Apply SBC to the Laplace approximation of a Poisson log-rate: $\theta \sim \mathcal{N}(0, 1)$, $n \sim \text{Poisson}(Te^\theta)$ with $T = 3$. Report the rank histogram, its χ^2, and 68% coverage. Interpret the shape of the histogram in terms of the true posterior when $n = 0$.

Identifiability and design

14. **[P] Nearly degenerate is not degenerate.** Observables $y_1 = \theta_1 + \theta_2$ and $y_2 = \theta_1 + 1.05\,\theta_2$, each with $\sigma = 0.1$. Compute $\mathcal{I}$, its eigenvalues and condition number, and the Cramér–Rao standard deviations of θ_1 and θ_2. Is the model structurally identifiable? Practically?

15. **[P] Fisher for counts.** You count $N \sim \text{Poisson}(\theta\,T)$ photons in exposure time T. Find $\mathcal{I}(\theta)$ and the Cramér–Rao bound. Does the MLE attain it? How does the bound change if T doubles?

16. **[P] Equivalent designs.** With $\theta \sim \mathcal{N}(0, 1)$, compare the EIG of: (a) one measurement $y = 3\theta + \epsilon$, $\sigma = 1$; (b) two independent measurements $y = \theta + \epsilon$, $\sigma = 1$; (c) one measurement $y = \theta + \epsilon$, $\sigma = 1/3$. Which are equivalent, and why?

17. **[R] Reading a Fisher matrix honestly.** A student computes the Fisher matrix for $(\alpha_{\text{MLT}}, f_{\text{ov}})$ from asteroseismic data and writes "the stiffest eigenvector is almost pure f_{ov}, so these data mainly constrain overshooting". What question would you ask first? What calculation would settle it?

Differentiable computing

18. **[P] Implicit and adjoint derivatives.** For $R(s,\theta) = s^3 + s - \theta = 0$: find $\mathrm{d}s/\mathrm{d}\theta$ at $\theta = 2$ by the implicit function theorem, then $\mathrm{d}\mathcal{L}/\mathrm{d}\theta$ for $\mathcal{L} = \frac{1}{2}s^2$ by the adjoint equation. Verify against the explicit solution near $\theta = 2$. Contrast with $R = s^2 - \theta$ as $\theta \to 0^+$.

19. **[C] Make the event differentiable.** Rewrite `mass_lost_event` in Listing 9 so that it does *not* use the closed form $t_s = \ln L_c/\theta$: locate the event numerically with a derivative-free root finder and obtain its derivative with `jax.lax.custom_root`. Confirm the gradient to 10 digits.

20. **[C] A score-function gradient.** For the IMF of Listing 10, estimate $\mathrm{d}P(M > 8\,M_\odot)/\mathrm{d}\gamma$ with the score-function estimator using $\nabla_\gamma \log p(m \mid \gamma)$. Compare its value and its standard error with the exact derivative and with the sigmoid relaxation. How many samples does the score-function estimator need for 5% relative error?

Research judgement

21. **[R] Critique an emulator claim.** A neural network reports 0.1% test error on stellar luminosity, with timesteps from the same tracks split at random between training and test sets. Explain why this may overstate generalization, and design a stronger evaluation, including what you would test if the emulator is later used inside HMC.

22. **[R] Formulate a discovery claim.** You want to constrain an exotic cooling parameter g from TRGB observables in globular clusters. Identify: (i) the target; (ii) at least four nuisance parameters or uncertainty sources; (iii) a likelihood including selection; (iv) an identifiability test; (v) an inference check; (vi) one independent validation.

B Worked solutions

1. $\mathcal{L}' = 4\theta(\theta^2 - 1)$, so the stationary points are $\theta = 0, \pm 1$. $\mathcal{L}'' = 12\theta^2 - 4$ is -4 at 0 (a local maximum) and $+8$ at ± 1 (minima). Hence $\min \mathcal{L} = -1$ and $\arg\min \mathcal{L} = \{-1, +1\}$, a two-element set. Started at exactly 0, the gradient vanishes and most libraries stop immediately and report convergence, at a *maximum*. From 10^{-6}, descent reaches $+1$, and from -10^{-6} it reaches -1. A symmetry of the model makes the best fit a set; report the set or the symmetry-invariant quantity (θ^2 here), and use several starts.

2. $C \int_1^{10} m^{-2}\mathrm{d}m = 0.9\,C = 1$ gives $C = 10/9$. $P(M > 5) = \frac{10}{9}(\frac{1}{5} - \frac{1}{10}) = 1/9 \approx 0.111$. The CDF is $F(m) = (1 - 1/m)/0.9$, so the median solves $1 - 1/m = 0.45$: $m_{1/2} = 20/11 \approx 1.82\,M_\odot$. The mean is $C \int_1^{10} m^{-1}\mathrm{d}m = \frac{10}{9}\ln 10 \approx 2.56\,M_\odot$. The mode is $1\,M_\odot$ (the density decreases). Mode $<$ median $<$ mean, the ordering for a right-skewed distribution: a few massive stars pull the mean up.

3. With $p_M(m) = 1/[m \ln(m_{\mathrm{hi}}/m_{\mathrm{lo}})]$, $m = L^{1/3.5}$ and $\mathrm{d}m/\mathrm{d}L = \frac{1}{3.5}L^{1/3.5-1}$:

$$p_L(L) = \frac{1}{L^{1/3.5}\ln(m_{\mathrm{hi}}/m_{\mathrm{lo}})} \cdot \frac{L^{1/3.5-1}}{3.5} = \frac{1}{L\,\ln(L_{\mathrm{hi}}/L_{\mathrm{lo}})},$$

using $\ln(L_{\mathrm{hi}}/L_{\mathrm{lo}}) = 3.5\ln(m_{\mathrm{hi}}/m_{\mathrm{lo}})$. L is log-uniform too: $\log L = 3.5 \log M$ is a linear map of a uniform variable. Log-uniform distributions are the scale-free ones, and every power law maps them to themselves.

4. (a) $f'(1) = -2.5$: $\mathbb{E}[t] \approx 1$, $\mathrm{sd}(t) \approx 0.25$ (in units of the solar-mass lifetime). (b) $f''(1) = 8.75$: $\mathbb{E}[t] \approx 1 + \frac{1}{2}(8.75)(0.01) = 1.044$. (c) No. The Gaussian density near $X = 0$ is tiny but positive, and $\int_0 x^{-2.5}\mathrm{d}x$ diverges, so $\mathbb{E}[X^{-2.5}]$ is infinite (and $X < 0$ gives non-real values). But $X = 0$ lies 10σ below the mean, so the probability of a draw near it is $\sim \Phi(-10) \approx 8 \times 10^{-24}$, and a Monte Carlo run with 10^6 draws returns a stable-looking ≈ 1.05. The finite answer is a property of the sample, not of the model: the input distribution allows unphysical masses. (d) For $\log X \sim \mathcal{N}(0, 0.1^2)$, exactly $\mathbb{E}[X^{-2.5}] = e^{\frac{1}{2}(2.5)^2(0.01)} = 1.0317$. The second-order formula with $\mathbb{E}X = e^{0.005}$ and $\mathrm{Var}\,X = (e^{0.01} - 1)e^{0.01}$ gives 1.0310. A positive input model gives a finite, well-behaved answer.

5. $\mathrm{Var}(X_1 + X_2) = 4 + 9 + 2(-3) = 7$; $\mathrm{Var}(X_1 - X_2) = 4 + 9 + 6 = 19$. Example: fitting a cluster turnoff, age and metallicity errors are anticorrelated (a more metal-rich model must be younger to match the same turnoff colour). The errors then scatter along the degeneracy line, on which a higher metallicity is paired with a lower age. The combination that moves *along* that line (age up, metallicity down) is poorly known. The combination that sums them with the appropriate weights, the one the turnoff colour actually measures, is well known: that is the analogue of $\mathrm{Var}(X_1 + X_2) = 7 < 19$.

6. Require $\sqrt{0.01 + 0.04/N} \le 0.11$, i.e. $0.04/N \le 0.0021$, so $N \ge 19.05$: **20 stars**. Beyond that, only information about θ itself helps: an independent calibrator, a different kind of observable sensitive to the offset, or a physical model that predicts it.

7. (a) $\eta < 2/\lambda_{\max} = 0.04$. (b) $\eta = 2/51 \approx 0.0392$, contraction $49/51 \approx 0.961$. (c) $\ln 10^6 / \ln(51/49) \approx 345$ iterations. (d) One. (e) $z_1 = \theta_1$, $z_2 = \sqrt{50}\,\theta_2$ gives $\mathcal{L} = \frac{1}{2}\|\mathbf{z}\|^2$, and gradient descent with $\eta = 1$ lands on the minimum in one step.

8. With $m_i = e^a e^{-t_i/e^b}$ and $r_i = (m_i - d_i)/\sigma_i$: $\partial r_i/\partial a = m_i/\sigma_i$, so $\partial^2 r_i/\partial a^2 = m_i/\sigma_i = \partial r_i/\partial a$ and $\partial^2 r_i/\partial a\,\partial b = \partial r_i/\partial b$. The extra Hessian terms in Eq. (17) are therefore $\sum_i r_i\,\partial r_i/\partial a = \partial\mathcal{L}/\partial a$ and $\sum_i r_i\,\partial r_i/\partial b = \partial\mathcal{L}/\partial b$, both zero at the optimum. $\partial^2 r_i/\partial b^2$ has no such relation, so the (b, b) entries differ.

9.

```python
"""Solution: Poisson NLL vs chi^2 with sigma_i = sqrt(n_i) (Neyman's chi^2), 500 data sets."""
import numpy as np
import jax
import jax.numpy as jnp
from scipy.optimize import minimize

jax.config.update("jax_enable_x64", True)
t = jnp.linspace(0.0, 4.0, 12)
a_true, b_true = jnp.log(8.0), jnp.log(1.5)
mu = lambda th: jnp.exp(th[0] - t / jnp.exp(th[1]))


def poisson_nll(th, n):
    return jnp.sum(mu(th) - n * jnp.log(mu(th)))


def neyman_chi2(th, n):
    return 0.5 * jnp.sum((mu(th) - n) ** 2 / jnp.maximum(n, 1.0))  # sigma^2 = n (n=0 -> 1)


def fit(loss, n):
    f = jax.jit(jax.value_and_grad(lambda th: loss(th, n)))
    res = minimize(lambda th: [np.asarray(v) for v in f(jnp.asarray(th))],
                   x0=np.array([1.0, 0.0]), jac=True, method="L-BFGS-B")
```

```
    return res.x


keys = jax.random.split(jax.random.key(9), 500)
fits = {"Poisson NLL": [], "Neyman chi^2": []}
for k in keys:
    n = jax.random.poisson(k, mu(jnp.array([a_true, b_true]))).astype(jnp.float64)
    fits["Poisson NLL"].append(fit(poisson_nll, n))
    fits["Neyman chi^2"].append(fit(neyman_chi2, n))
for name, f in fits.items():
    f = np.array(f)
    print(f"{name:>12}: mean(a_hat - a_true) = {f[:, 0].mean() - float(a_true):+.4f}"
          f" +/- {f[:, 0].std() / np.sqrt(len(f)):.4f};  sd(a_hat) = {f[:, 0].std():.4f}")
```

Output (executed for these notes):

```
  Poisson NLL: mean(a_hat - a_true) = -0.0052 +/- 0.0106;  sd(a_hat) = 0.2374
Neyman chi^2: mean(a_hat - a_true) = -0.1076 +/- 0.0142;  sd(a_hat) = 0.3176
```

The Poisson fit is unbiased within its Monte Carlo error. Neyman's χ^2 with $\sigma_i^2 = n_i$ underestimates the amplitude by about 0.11 (7 standard errors) and is less precise. (Empty bins, $n_i = 0$, would get zero variance; the listing uses $\sigma_i^2 = \max(n_i, 1)$, one common patch, which is itself a sign that this loss is not derived from the noise model.) Its weights $1/n_i$ are largest for bins that fluctuated *low*, so the fit is pulled down.

10. Precisions add: $1 + 1/1 + 9/4 = 4.25$, so Var $= 0.235$, and $\mu = 0.235\,(0.8/1 + 3 \times 1.5/4) = 0.453$. Sequentially: after y_1 the posterior is $\mathcal{N}(0.4, 0.5)$, precision 2; using it as the prior for y_2 gives precision $2 + 9/4 = 4.25$ and mean $(0.4 \times 2 + 1.125)/4.25 = 0.453$. Bayesian updating with independent data is order-independent.

11. In θ: $\mathrm{d}\log p/\mathrm{d}\theta = (k-1)/\theta - 1/s = 0$ gives $\theta_{\mathrm{MAP}} = (k-1)s$. In z, the density gains a factor $|\mathrm{d}\theta/\mathrm{d}z| = \theta$, so $p(z) \propto \theta^k e^{-\theta/s}$, with maximum at $\theta = ks$. For $k = 3$, $s = 1$: 2 versus 3. The median (≈ 2.674 in both coordinates) is invariant because quantiles are preserved by monotone maps.

12. Measured with Listing 4 (acceptance, ESS of θ_1): $\varepsilon = 0.05$: 0.99, 443; 0.15: 0.96, $\approx 10^4$; 0.25: 0.72, ≈ 6700; 0.30 and 0.45: *zero* acceptance. The precision matrix has eigenvalues $1/(1-\rho) = 50$ and $1/(1+\rho) \approx 0.505$. In the stiff direction the dynamics are a harmonic oscillator with $\omega = \sqrt{50} \approx 7.07$, and leapfrog is stable only for $\varepsilon\omega < 2$, i.e. $\varepsilon < 0.283$. Beyond it the energy error grows exponentially and every proposal is rejected. This is the same lesson as the gradient-descent limit $\eta < 2/\lambda_{\max}$: the stiffest direction caps the step (for leapfrog, $\varepsilon < 2/\sqrt{\lambda_{\max}}$). The smallest step is stable but its trajectory ($20 \times 0.05 = 1$) is short compared with the time to cross the soft direction ($\sim \pi/0.71 \approx 4.4$), so successive draws are correlated. A mass matrix equal to the precision would remove the stiffness, the sampling analogue of the rescaling in Section 2.6.

13.

```
"""Solution to the SBC-of-Laplace exercise: Poisson counts with a log-rate parameter."""
import jax
import jax.numpy as jnp

jax.config.update("jax_enable_x64", True)
T, n_sims, S = 3.0, 4000, 99


def laplace(n):
    U = lambda th: -(n * th - T * jnp.exp(th)) + 0.5 * th**2  # negative log posterior
    th = jnp.log((n + 0.5) / T)
```

```
    for _ in range(30):  # Newton iterations on the scalar U
        th = th - jax.grad(U)(th) / jax.grad(jax.grad(U))(th)
    return th, 1.0 / jnp.sqrt(jax.grad(jax.grad(U))(th))


def one(key):
    k1, k2, k3 = jax.random.split(key, 3)
    th = jax.random.normal(k1)
    n = jax.random.poisson(k2, T * jnp.exp(th))
    m, s = laplace(n)
    draws = m + s * jax.random.normal(k3, (S,))
    return jnp.sum(draws < th), jnp.abs(th - m) < s, n


ranks, cov, n = jax.vmap(one)(jax.random.split(jax.random.key(3), n_sims))
counts = jnp.histogram(ranks, bins=10, range=(0, S + 1))[0]
print("rank counts:", counts)
print(f"chi^2 = {jnp.sum((counts - n_sims / 10) ** 2 / (n_sims / 10)):.1f} (9 dof)")
print(f"68% coverage overall = {cov.mean():.3f}; for n=0: {cov[n == 0].mean():.3f}"
      f" ({int((n == 0).sum())} sims); for n>=10: {cov[n >= 10].mean():.3f}")
```

Output (executed for these notes):

```
rank counts: [545. 438. 411. 375. 371. 404. 421. 419. 363. 253.]
chi^2 = 119.6 (9 dof)
68% coverage overall = 0.686; for n=0: 0.690 (548 sims); for n>=10: 0.695
```

Coverage is nominal, yet the rank histogram fails ($\chi^2 \approx 120$ for 9 degrees of freedom): too many low ranks and too few in the top bin. For $n = 0$ the true posterior $\propto \exp(-Te^\theta - \theta^2/2)$ has a long tail toward negative θ and a sharp edge at positive θ. The symmetric Laplace approximation puts too much mass on the right and too little in the left tail, so a truth from the left tail lands below nearly all draws.

14. $\mathcal{I} = 100\,\mathbf{J}^\mathsf{T}\mathbf{J} = \left(\begin{smallmatrix} 200 & 205 \\ 205 & 210.25 \end{smallmatrix}\right)$, eigenvalues 0.061 and 410.2, condition number ≈ 6700. $\mathcal{I}^{-1} = \left(\begin{smallmatrix} 8.41 & -8.20 \\ -8.20 & 8.00 \end{smallmatrix}\right)$, so the Cramér–Rao standard deviations are 2.90 and 2.83, thirty times the measurement errors. The model is structurally identifiable ($\mathbf{J}$ has full rank) but practically unidentifiable at this precision: the weak direction is constrained only to $\pm 1/\sqrt{0.061} \approx \pm 4$.

15. $\ell = N\log(\theta T) - \theta T - \log N!$, so the score is $N/\theta - T$ and $\mathcal{I} = \mathbb{E}[N]/\theta^2 = T/\theta$. The bound is $\mathrm{Var}(\hat\theta) \geq \theta/T$. The MLE $\hat\theta = N/T$ is unbiased with variance $\theta T/T^2 = \theta/T$, so it attains the bound. Doubling T halves it.

16. (a) $\frac{1}{2}\ln(1+9) = 1.151$ nats; (b) the Fisher information is $1 + 1 = 2$, so $\frac{1}{2}\ln 3 = 0.549$; (c) $\mathcal{I} = 9$, $\frac{1}{2}\ln 10 = 1.151$. (a) and (c) are equivalent: in the linear-Gaussian model the EIG depends on the data only through $\mathcal{I} = a^2/\sigma^2$. Amplifying the signal threefold is worth the same as cutting the noise threefold, which is worth nine repeated measurements, not two.

17. First ask: in what units are the two parameters expressed, and how wide are their priors? A Fisher eigenvector is not unit-invariant (Section 4.2.1). Settle it with the prior-whitened generalized eigenproblem Eq. (34): report the eigenvalues relative to the prior and the combinations they correspond to. Then check the linear picture with a profile likelihood or posterior samples.

18. $\mathrm{d}s/\mathrm{d}\theta = -R_\theta/R_s = 1/(3s^2+1)$. At $\theta = 2$, $s = 1$, so $\mathrm{d}s/\mathrm{d}\theta = 1/4$. Adjoint: $R_s\lambda = \partial\mathcal{L}/\partial s = s = 1$ gives $\lambda = 1/4$, and $\mathrm{d}\mathcal{L}/\mathrm{d}\theta = -\lambda R_\theta = 1/4$. Check: at $\theta = 2.01$, $s \approx 1.0025$ and $\mathcal{L}$ rises by ≈ 0.0025, a slope of 0.25. For $R = s^2 - \theta$, $R_s = 2s \to 0$ as $\theta \to 0^+$ and $\mathrm{d}s/\mathrm{d}\theta = 1/(2\sqrt{\theta}) \to \infty$: the two branches $s = \pm\sqrt{\theta}$ merge, a bifurcation. Finite differences straddling $\theta = 0$ mix branches or leave the domain.

19.

```python
"""Solution to the event exercise: locate t_s by a root find and let JAX apply
the implicit function theorem, without using the closed form t_s = ln(L_c)/theta."""
import jax
import jax.numpy as jnp

jax.config.update("jax_enable_x64", True)
L_c, rate, t_end = 10.0, 1.0, 5.0


def event_time(theta):
    g = lambda t: jnp.exp(theta * t) - L_c  # event function: zero at the switch-on time

    def bisect(g, t0, lo=0.0, hi=t_end, iters=60):  # robust, derivative-free forward solve
        def body(_, lohi):
            lo, hi = lohi
            mid = 0.5 * (lo + hi)
            return jnp.where(g(mid) > 0, lo, mid), jnp.where(g(mid) > 0, mid, hi)
        lo, hi = jax.lax.fori_loop(0, iters, body, (lo, hi))
        return 0.5 * (lo + hi)

    tangent_solve = lambda gl, y: y / gl(1.0)  # g is scalar: divide by its derivative
    return jax.lax.custom_root(g, 0.0, bisect, tangent_solve)


def mass_lost(theta):
    return rate * (t_end - event_time(theta))


th = 1.0
print(f"t_s        = {event_time(th):.12f}  (exact {jnp.log(L_c) / th:.12f})")
print(f"dDeltaM/dth = {jax.grad(mass_lost)(th):.12f}  (exact {rate * jnp.log(L_c) / th**2:.12f})")
```

Output (executed for these notes):

```
t_s        = 2.302585092994  (exact 2.302585092994)
dDeltaM/dth = 2.302585092994  (exact 2.302585092994)
```

The bisection is not differentiable, but it does not need to be. `custom_root` differentiates the event condition $g(t_s, \theta) = 0$ at the solution by the implicit function theorem.

20.

```python
"""Solution: score-function estimate of dP(M > 8 Msun)/dgamma for a power-law IMF."""
import jax
import jax.numpy as jnp
from ex02_imf_pathwise import inv_cdf, frac_above_exact, m_lo, m_hi, m_crit

jax.config.update("jax_enable_x64", True)
gamma = 2.35


def log_p(m, g):  # log density of the normalized power law
    k = 1.0 - g
    return jnp.log(k / (m_hi**k - m_lo**k)) - g * jnp.log(m)


N = 400_000
m = inv_cdf(jax.random.uniform(jax.random.key(21), (N,)), gamma)
score = jax.vmap(jax.grad(log_p, argnums=1), in_axes=(0, None))(m, gamma)
f = (m > m_crit).astype(jnp.float64)
```

```python
for name, terms in [("raw", f * score), ("with baseline", (f - f.mean()) * score)]:
    est, se = terms.mean(), terms.std() / jnp.sqrt(N)
    n5 = (terms.std() / (0.05 * abs(est))) ** 2
    print(f"score function ({name:>13}): {est:+.5f} +/- {se:.5f};"
          f"  N for 5% error ~ {n5:,.0f}")
print(f"exact: {jax.grad(frac_above_exact)(gamma):+.5f}")
```

Output (executed for these notes):

```
score function (          raw): -0.06155 +/- 0.00065;  N for 5% error ~ 17,868
score function (with baseline): -0.06156 +/- 0.00064;  N for 5% error ~ 17,055
exact: -0.06166
```

For the power law, $\nabla_\gamma \log p(m \mid \gamma) = -\ln m + \mathrm{d}\log C/\mathrm{d}\gamma$. The estimate agrees with the exact derivative within one standard error. About 1.8×10^4 draws give 5% relative error; a baseline barely helps here because the indicator is almost always zero. Compare: the pathwise estimator returns 0 at any sample size, and the sigmoid relaxation carries a small w-dependent bias (0.3% for $w = 0.25\,M_\odot$, by quadrature) that is hidden by its Monte Carlo error at this sample size.

21. Neighbouring timesteps on one track are nearly duplicates, so a random split tests interpolation along tracks already seen. Instead: hold out whole tracks; hold out mass and metallicity regions and evolutionary phases as separate stress tests; report the metrics that matter downstream (for example, maximum error near the TRGB, not mean error over the main sequence); test out-of-distribution behaviour and whether the emulator signals it. If the emulator will sit inside HMC, also check its *derivatives* against Stellax directional finite differences, check its behaviour across events, and include its error in the likelihood (Section 4.6). Finally, compare posteriors obtained with the emulator and with Stellax on a few simulated data sets, and run SBC with the emulator in the loop.

22. One defensible answer. (i) Target: the coupling g. (ii) Nuisances: conductive opacities and neutrino-loss rates (which set the core mass at the flash); helium abundance, [Fe/H] and [α/Fe]; distance modulus and extinction; bolometric corrections and photometric zero points; mass loss on the RGB. (iii) Likelihood: per cluster, the observed magnitudes of the brightest giants given a predicted tip luminosity. Model the sparse sampling near the tip (a cluster with few bright giants may have no star at the true tip), selection and crowding, and combine clusters hierarchically with shared g and shared physics. (iv) Identifiability: prior-whitened Fisher eigenvalues for (g, conductive opacity, distance); expect a degeneracy between g and distance that independent distances must break. (v) Inference check: SBC on simulated clusters. (vi) Validation: an independent observable sensitive to the same physics in a different way (for example, horizontal-branch to giant-branch number ratios), or a repeat with alternative opacity and neutrino-loss prescriptions. State the conclusion conditionally on the tested model family.

C Glossary

Acquisition function Ranks the next candidate in active learning or Bayesian optimization.

Adjoint method Gradient of a scalar through an implicit solve using one transposed linear solve (Eq. (40)).

Aleatoric uncertainty Variability of outcomes under a stated conditioning set.

Argmin / argmax The input(s) attaining the smallest / largest value; a set when there are ties.

Automatic differentiation (AD) The chain rule applied to the operations a program executes.

Bayesian optimization Sequential minimization of an expensive objective with a surrogate and an acquisition rule.

Calibration Physical: fitting a model to reference data. Probabilistic: agreement of stated probabilities with frequencies. Say which.

Complex step $f'(x) \approx \operatorname{Im} f(x + ih)/h$; machine-precision derivative for analytic code.

Condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ of a symmetric positive-definite matrix; sets gradient-descent speed and linear-solve accuracy.

Convexity Chords lie on or above the graph; tangent planes are global under-estimators.

Cramér–Rao bound $\operatorname{Cov}(\hat{\boldsymbol{\theta}}) \succeq \mathcal{I}^{-1}$ for unbiased estimators, under regularity conditions.

Credible / confidence interval Posterior probability statement / long-run coverage property of a procedure.

Degeneracy Different parameters giving indistinguishable predictions.

Delta method Uncertainty propagation by Taylor expansion: $\mathbf{J}\boldsymbol{\Sigma}\mathbf{J}^\mathsf{T}$ at first order.

Detailed balance $p(\boldsymbol{\theta})T(\boldsymbol{\theta} \to \boldsymbol{\theta}') = p(\boldsymbol{\theta}')T(\boldsymbol{\theta}' \to \boldsymbol{\theta})$; sufficient for p to be stationary.

Divergence (HMC) A trajectory whose energy error explodes; flags curvature the sampler cannot explore.

Effective sample size Number of independent draws with the same Monte Carlo error as a correlated chain.

Epistemic uncertainty Incomplete knowledge of parameters, mechanisms, or model adequacy.

Evidence $p(\mathbf{d})$: the data probability averaged over the prior; strongly prior-dependent.

Expected information gain Mutual information between parameters and planned data (Eq. (36)).

Fisher information Covariance of the score; $\mathbf{J}^\mathsf{T}\boldsymbol{\Sigma}^{-1}\mathbf{J}$ for Gaussian noise with fixed covariance.

Gauss–Newton Least-squares Hessian approximation $\mathbf{J}^\mathsf{T}\boldsymbol{\Sigma}^{-1}\mathbf{J}$.

Hamiltonian Monte Carlo MCMC that proposes by simulating Hamiltonian dynamics on $-\log p$.

Hierarchical model A model with population-level hyperparameters, individual latent variables, and shared physics.

Identifiability Structural: perfect data determine the parameters. Practical: the available data constrain them enough.

Implicit function theorem $\mathrm{d}\mathbf{s}/\mathrm{d}\boldsymbol{\theta} = -\mathbf{R}_s^{-1}\mathbf{R}_\theta$ for a state defined by $\mathbf{R}(\mathbf{s}, \boldsymbol{\theta}) = \mathbf{0}$.

JVP / VJP Jacobian–vector and vector–Jacobian products; forward- and reverse-mode AD.

Laplace approximation A Gaussian at the MAP with covariance equal to the inverse Hessian of $-\log p$.

Latent variable An unobserved quantity represented explicitly in a generative model.

Leakage Information about the test set reaching training, e.g. neighbouring timesteps of one track.

Levenberg–Marquardt Damped Gauss–Newton; interpolates between Gauss–Newton and scaled gradient descent.

Likelihood $p(\mathbf{d} \mid \boldsymbol{\theta})$ read as a function of $\boldsymbol{\theta}$; not a density in $\boldsymbol{\theta}$.

MAP / MLE Posterior mode (coordinate-dependent) / likelihood maximizer (reparameterization-invariant).

Marginalization / profiling Integrating out / maximizing over nuisance parameters.

Model discrepancy Systematic difference between model and reality not explained by parameters or noise.

Pathwise (reparameterization) gradient $\nabla\mathbb{E}[f(g(\boldsymbol{\theta}, \epsilon))] = \mathbb{E}[\nabla f(g(\boldsymbol{\theta}, \epsilon))]$; invalid when f is discontinuous.

Posterior predictive $p(\tilde{\mathbf{d}} \mid \mathbf{d})$: predictions averaged over posterior uncertainty.

Prior-whitened Fisher eigenvalues Solutions of $\mathcal{I}\mathbf{v} = \lambda \boldsymbol{\Sigma}_0^{-1}\mathbf{v}$; unit-invariant information relative to the prior.

Pushforward The distribution of $\mathcal{G}(\boldsymbol{\Theta})$ when $\boldsymbol{\Theta} \sim p$.

$\hat{R}$ A between-chain versus within-chain variance diagnostic; rank-normalized $\hat{R} < 1.01$ is the usual target.

Score $\nabla_{\boldsymbol{\theta}} \log p(\mathbf{d} \mid \boldsymbol{\theta})$.

Score-function estimator $\nabla \mathbb{E}_{p_\theta}[f] = \mathbb{E}_{p_\theta}[f \nabla \log p_\theta]$; unbiased for discontinuous f, often noisy.

Selection function Probability that an object enters the catalogue; must normalize population likelihoods (Eq. (31)).

Simulation-based calibration Rank test that an inference code returns calibrated posteriors.

Surrogate / emulator A cheap approximation of an expensive model.

Trust region Neighbourhood in which a local model of the objective is trusted when choosing a step.

Suggested reading

- **Fitting and probability in astronomy:** Hogg, Bovy & Lang [24]; Gelman et al. [25] for hierarchical models and model checking.
- **Optimization:** Nocedal & Wright [26]; Boyd & Vandenberghe [27] for convexity and duality; Kochenderfer & Wheeler [28] for an algorithmic survey (a second edition is announced by MIT Press for January 2027).
- **Sampling:** Betancourt [5] on HMC; Vehtari et al. [7] on $\hat{R}$ and ESS; Talts et al. [11] on SBC.
- **Gradients of expectations and through solvers:** Mohamed et al. [23]; Blondel et al. [21]; Baydin et al. [20]; Blondel & Roulet [29], a book-length treatment of differentiable programming.
- **Machine learning for astronomy:** Ting [30], a statistical-ML textbook written for astronomers; *Deep Learning for Astrophysics* [31], a hands-on textbook from neural-network foundations to generative models and simulation-based inference; Upendran & Díaz Baso [32] on machine learning for the solar interior and atmosphere.
- **Connections across fields:** Blumenthal et al. [33] on the shared mathematics of control, inference, optimal transport and thermodynamics, with applications to reinforcement learning, variational inference and generative modelling.
- **Identifiability and design:** Raue et al. [13]; Rainforth et al. [17]; Kennedy & O'Hagan [14] and Brynjarsdóttir & O'Hagan [15] on discrepancy.
- **Surrogates:** Rasmussen & Williams [18]; Cranmer, Brehmer & Louppe [34] on simulation-based inference.
- **Population inference with selection:** Loredo [8]; Mandel, Farr & Gair [9].

References

[1] E. E. Salpeter. The luminosity function and stellar evolution. *The Astrophysical Journal*, 121: 161, 1955.

[2] P. Kroupa. On the variation of the initial mass function. *Monthly Notices of the Royal Astronomical Society*, 322:231–246, 2001.

[3] G. Chabrier. Galactic stellar and substellar initial mass function. *Publications of the Astronomical Society of the Pacific*, 115:763–795, 2003.

[4] R. M. Neal. MCMC using Hamiltonian dynamics. In S. Brooks, A. Gelman, G. L. Jones, and X.-L. Meng, editors, *Handbook of Markov Chain Monte Carlo*, pages 113–162. Chapman & Hall/CRC, 2011.

[5] M. Betancourt. A conceptual introduction to Hamiltonian Monte Carlo. arXiv:1701.02434, 2017.

[6] M. D. Hoffman and A. Gelman. The No-U-Turn sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15:1593–1623, 2014.

[7] A. Vehtari, A. Gelman, D. Simpson, B. Carpenter, and P.-C. Bürkner. Rank-normalization, folding, and localization: An improved $\widehat{R}$ for assessing convergence of MCMC. *Bayesian Analysis*, 16(2):667–718, 2021.

[8] T. J. Loredo. Accounting for source uncertainties in analyses of astronomical survey data. In *Bayesian Inference and Maximum Entropy Methods in Science and Engineering*, volume 735 of *AIP Conference Proceedings*, pages 195–206, 2004.

[9] I. Mandel, W. M. Farr, and J. R. Gair. Extracting distribution parameters from multiple uncertain observations with selection biases. *Monthly Notices of the Royal Astronomical Society*, 486:1086–1093, 2019.

[10] A. Gelman, X.-L. Meng, and H. Stern. Posterior predictive assessment of model fitness via realized discrepancies. *Statistica Sinica*, 6:733–807, 1996.

[11] S. Talts, M. Betancourt, D. Simpson, A. Vehtari, and A. Gelman. Validating Bayesian inference algorithms with simulation-based calibration. arXiv:1804.06788, 2018.

[12] T. J. Rothenberg. Identification in parametric models. *Econometrica*, 39(3):577–591, 1971.

[13] A. Raue, C. Kreutz, T. Maiwald, J. Bachmann, M. Schilling, U. Klingmüller, and J. Timmer. Structural and practical identifiability analysis of partially observed dynamical models by exploiting the profile likelihood. *Bioinformatics*, 25(15):1923–1929, 2009.

[14] M. C. Kennedy and A. O'Hagan. Bayesian calibration of computer models. *Journal of the Royal Statistical Society: Series B*, 63(3):425–464, 2001.

[15] J. Brynjarsdóttir and A. O'Hagan. Learning about physical parameters: the importance of model discrepancy. *Inverse Problems*, 30:114007, 2014.

[16] T. Rainforth, R. Cornish, H. Yang, A. Warrington, and F. Wood. On nesting Monte Carlo estimators. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *PMLR*, pages 4267–4276, 2018.

[17] T. Rainforth, A. Foster, D. R. Ivanova, and F. Bickford Smith. Modern Bayesian experimental design. *Statistical Science*, 39(1):100–114, 2024.

[18] C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

[19] M. C. Kennedy and A. O'Hagan. Predicting the output from a complex computer code when fast approximations are available. *Biometrika*, 87(1):1–13, 2000.

[20] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018.

[21] M. Blondel, Q. Berthet, M. Cuturi, R. Frostig, S. Hoyer, F. Llinares-López, F. Pedregosa, and J.-P. Vert. Efficient and modular implicit differentiation. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[22] W. Squire and G. Trapp. Using complex variables to estimate derivatives of real functions. *SIAM Review*, 40(1):110–112, 1998.

[23] S. Mohamed, M. Rosca, M. Figurnov, and A. Mnih. Monte Carlo gradient estimation in machine learning. *Journal of Machine Learning Research*, 21(132):1–62, 2020.

[24] D. W. Hogg, J. Bovy, and D. Lang. Data analysis recipes: Fitting a model to data. arXiv:1008.4686, 2010.

[25] A. Gelman, J. B. Carlin, H. S. Stern, D. B. Dunson, A. Vehtari, and D. B. Rubin. *Bayesian Data Analysis*. CRC Press, 3rd edition, 2013.

[26] J. Nocedal and S. J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.

[27] S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[28] M. J. Kochenderfer and T. A. Wheeler. *Algorithms for Optimization*. MIT Press, 2019.

[29] M. Blondel and V. Roulet. The elements of differentiable programming. arXiv:2403.14606, 2024. URL https://diffprog.github.io/.

[30] Y.-S. Ting. Statistical machine learning for astronomy. arXiv:2506.12230, 2025. URL https://tingyuansen.github.io/statml/.

[31] NASA Cosmic Origins AI/ML Science and Technology Interest Group. Deep learning for astrophysics. Online textbook. URL https://deeplearning4astro.com/. Chapters by Y.-S. Ting, C. Cuesta-Lazaro, A. Scaife, J. F. Wu and others.

[32] V. Upendran and C. J. Díaz Baso. Machine learning applications in the solar interior and atmosphere. arXiv:2610.34687, 2026. URL https://arxiv.org/abs/2610.34687.

[33] E. Blumenthal, N. Claussen, B. Eysenbach, C. Ji, G. Reddy, C. Scheibner, and B. Sorkin. Bridging control, inference, transport, and thermodynamics: From theory to applications in learning. arXiv:2609.15897, 2026. URL https://arxiv.org/abs/2609.15897.

[34] K. Cranmer, J. Brehmer, and G. Louppe. The frontier of simulation-based inference. *Proceedings of the National Academy of Sciences*, 117(48):30055–30062, 2020.